WealthWise

Intelligent Portfolio Analytics & Goal-Planning Platform

Complete Documentation Suite

Faishal Rahman Ansari
Genesis Softwares
VTU Internship 2026 · Team 11

April 2026

Version 2.0

Table of Contents

WealthWise Complete Documentation Suite — 8 Documents

01 Project Synopsis §1
02 Software Requirements Specification §2
03 System Design: HLD & LLD §3
04 API Documentation §4
05 Deployment Guide §5
06 User Manual §6
07 Testing Documentation §7
08 Internship Progress Report §8

DOCUMENT 01

Project Synopsis

WealthWise Documentation Suite · April 2026

WealthWise — Project Synopsis

1. Project Title

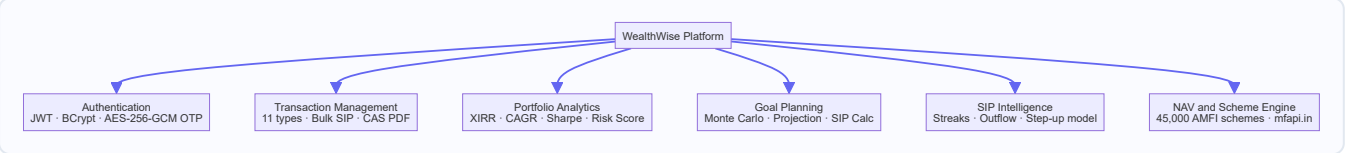
WealthWise: An Intelligent Portfolio Analytics and Goal-Planning Platform for Indian Mutual Fund Investors

2. Abstract

WealthWise is an enterprise-grade, full-stack web application for Indian retail mutual fund investors. It consolidates portfolio data from manual entry and CAS (Consolidated Account Statement) PDF imports, then delivers institutional-quality analytics: XIRR, Sharpe ratio, SEBI risk profiling, SIP intelligence, goal-based planning with Monte Carlo simulation, and a capital gains tax report.

The platform is built on **React 19 + Vite 8** (frontend), **Spring Boot 3.2.3** (backend), **PostgreSQL 15 via Supabase** (database), and deployed via **Docker on Render**.

3. Platform Feature Overview



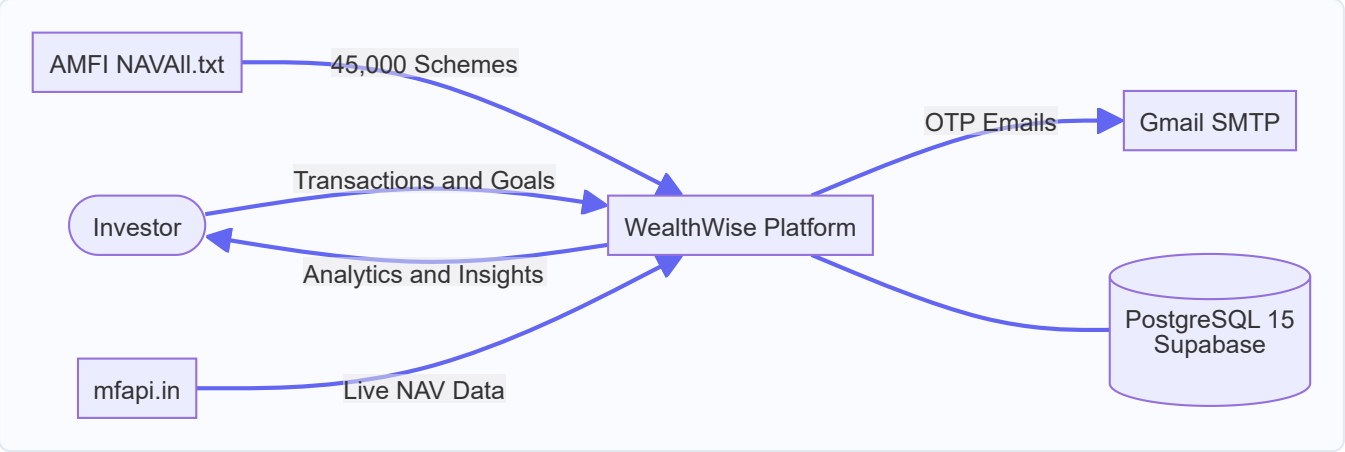
4. Problem Statement

> *Indian retail mutual fund investors lack a unified, intelligent platform that consolidates transaction data, provides institutional analytics (XIRR, risk profiling, tax computation), enables probabilistic goal planning, and does all of this with the security standards required for financial applications.*

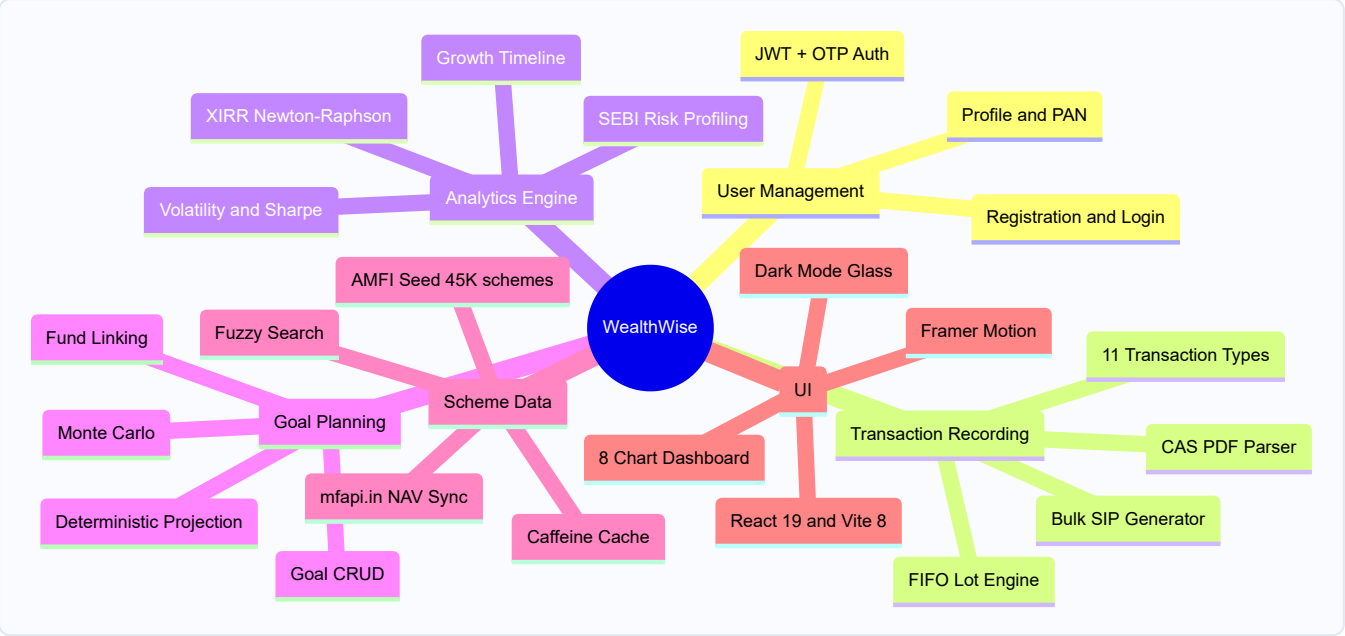
5. Objectives

ID	Objective	Success Metric
O1	Unified Portfolio View	All holdings visible within 3 seconds
O2	Automated CAS Ingestion	≥ 95% parse accuracy
O3	Institutional Analytics	XIRR, Sharpe ratio, volatility with < 1% deviation
O4	SIP Intelligence	Streaks, step-up modelling for 10+ years
O5	Probabilistic Goal Planning	Monte Carlo 10,000 paths; P10/P50/P90
O6	Enterprise Security	BCrypt + JWT + AES-256-GCM PAN + HSTS
O7	Production Deployment	Docker on Render; cold start ≤ 90s

6. System Context



7. Application Scope



8. Technology Stack

8.1 Full Stack Overview

8.2 Frontend Technologies

Technology	Version	Purpose
React	19.2.4	Concurrent rendering; SPA framework
Vite	8.0.4	Fast native ESM builds
React Router DOM	7.14.0	SPA routing with protected routes
Recharts	3.8.1	Animated data charts on D3
Framer Motion	12.38.0	Spring physics page transitions
Lucide React	Latest	Tree-shakeable icon library

8.3 Backend Technologies

Technology	Version	Purpose
Java	17 (LTS)	Required by Spring Boot 3.x
Spring Boot	3.2.3	Auto-configured REST API
Spring Security	6.x	JWT filter chain; BCrypt
Spring Data JPA	3.x	Repository pattern; type-safe queries
Apache PDFBox	3.0.1	Pure Java CAS PDF parsing
JJWT	0.12.5	HMAC-SHA256 JWT tokens
Caffeine Cache	Latest	In-process cache (24h NAV TTL)

9. Key Technical Innovations

Innovation	Description
XIRR via Newton-Raphson	Custom Java implementation on lot-level cash flows; converges in < 100 iterations
FIFO Lot Engine	Every purchase creates an `InvestmentLot`; FIFO disposal on redemption with STCG/LTCG split
CAS PDF Pipeline	PDFBox → regex → AMFI fuzzy reconciliation → bulk insert
Monte Carlo Engine	10,000 paths; P10/P50/P90 percentiles; inflation-adjusted target
Configurable Dashboard	8 chart types; localStorage preference persistence
Portfolio Growth Timeline	Server-side monthly snapshot from transaction history

10. Project Team

Role	Name	Contribution
Frontend and Analytics	Faishal Rahman Ansari	Portfolio page, Goals system, Dashboard charts, SIP Intelligence, Testing
Organization	Genesis Softwares	Mentorship, infrastructure, code reviews
Program	VTU Internship 2026 — Team 11	Degree internship project

DOCUMENT 02

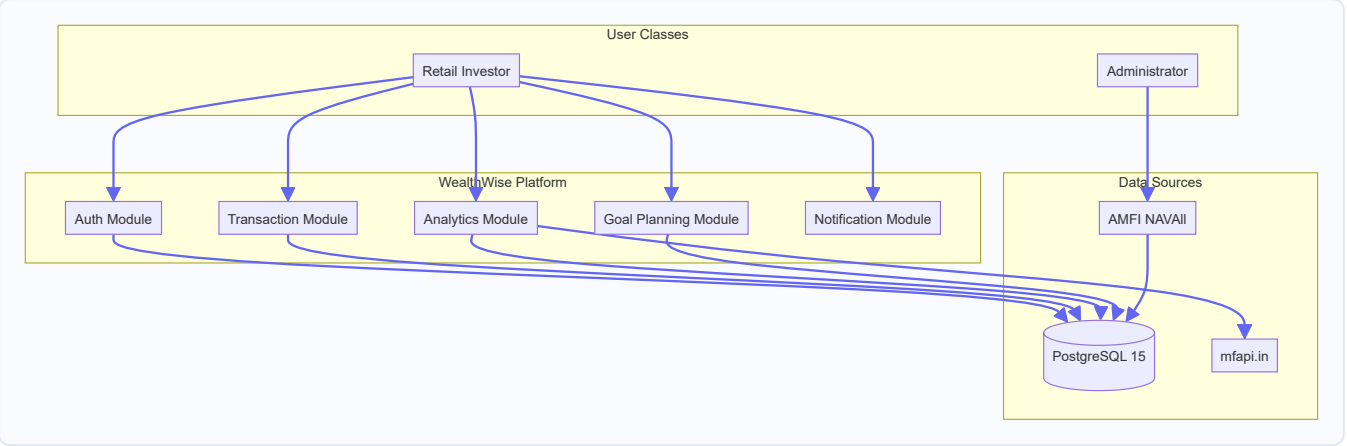
Software Requirements Specification

WealthWise Documentation Suite · April 2026

WealthWise — Software Requirements Specification (SRS)

Version: 2.0 | **Date:** April 2026
Prepared by: Faishal Rahman Ansari | **Organization:** Genesis Softwares

1. System Overview



1.1 Document Scope

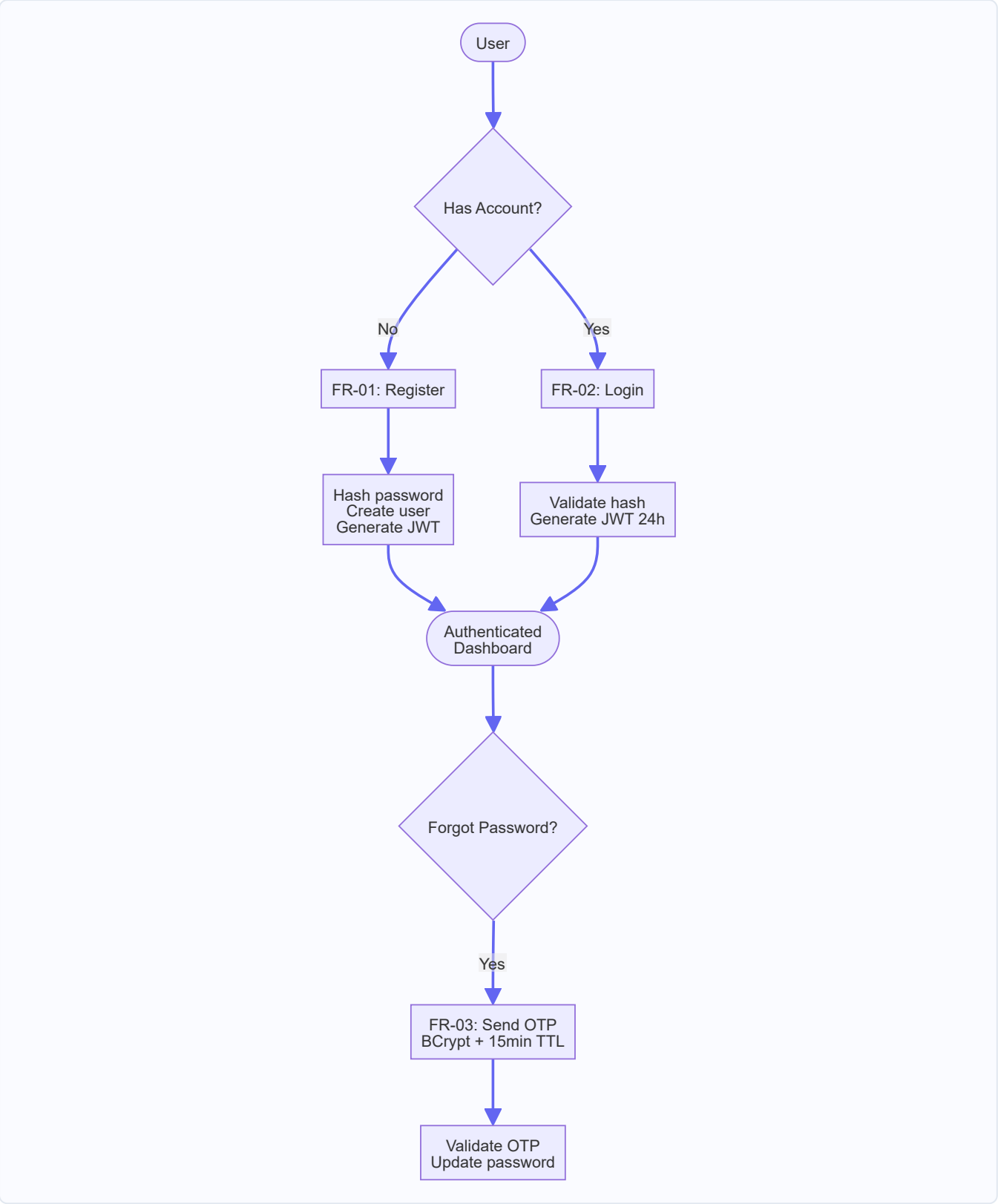
This SRS covers the complete WealthWise web application: React 19/Vite 8 SPA, Spring Boot 3.2.3 REST API, PostgreSQL 15 database, JWT authentication, CAS PDF parsing, and portfolio analytics/goal-planning engine.

1.2 Definitions

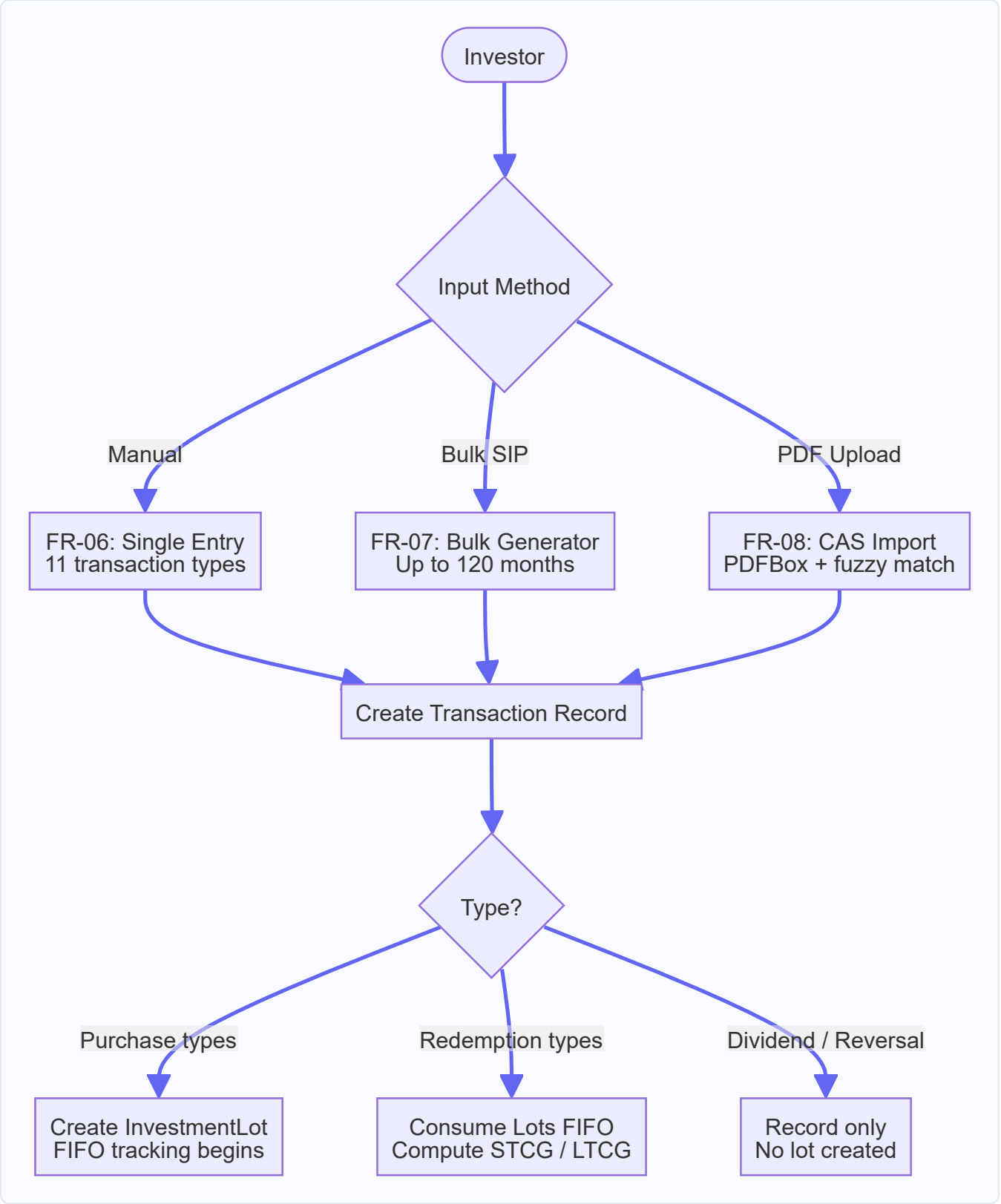
Term	Definition
SIP	Systematic Investment Plan — periodic fixed-amount MF investment
CAS	Consolidated Account Statement — SEBI-mandated doc from CAMS/KFintech
NAV	Net Asset Value — price per unit of a mutual fund scheme
XIRR	Extended Internal Rate of Return — IRR for irregular cash flows
AMFI	Association of Mutual Funds in India
FIFO	First In, First Out — lot disposal order for redemption tracking
LTCG	Long-Term Capital Gain (held ≥ 1 year)
STCG	Short-Term Capital Gain (held < 1 year)

2. Functional Requirements

2.1 Authentication Flow



2.2 Transaction Management Flow

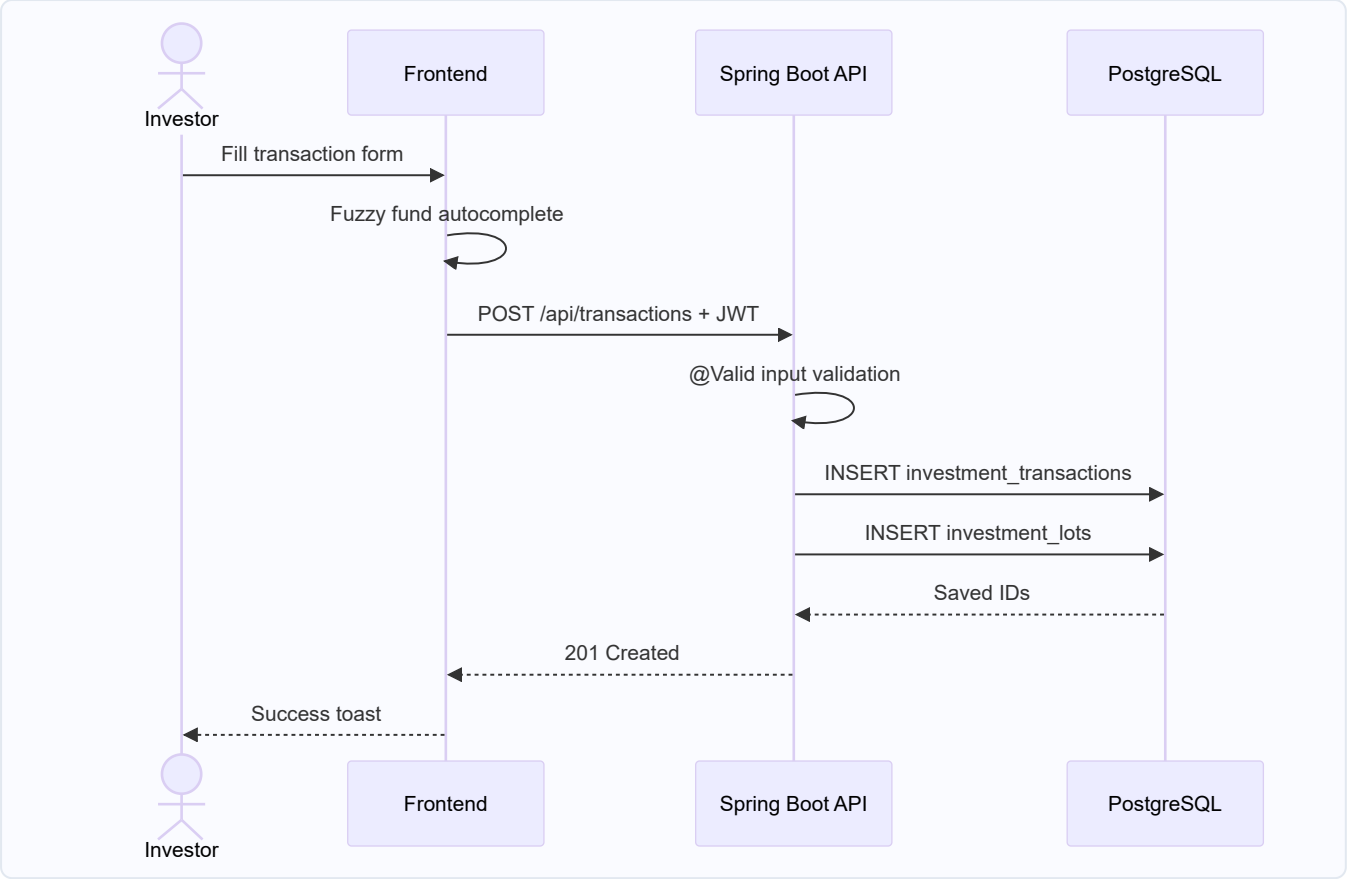

Investor
Input Method
Manual
Bulk SIP
PDF Upload
FR-06: Single Entry
11 transaction types
FR-07: Bulk Generator
Up to 120 months
FR-08: CAS Import
PDFBox + fuzzy match
Create Transaction Record
Type?
Purchase types
Redemption types
Dividend / Reversal
Create InvestmentLot
FIFO tracking begins
Consume Lots FIFO
Compute STCG / LTCG
Record only
No lot created

2.3 Functional Requirements Table

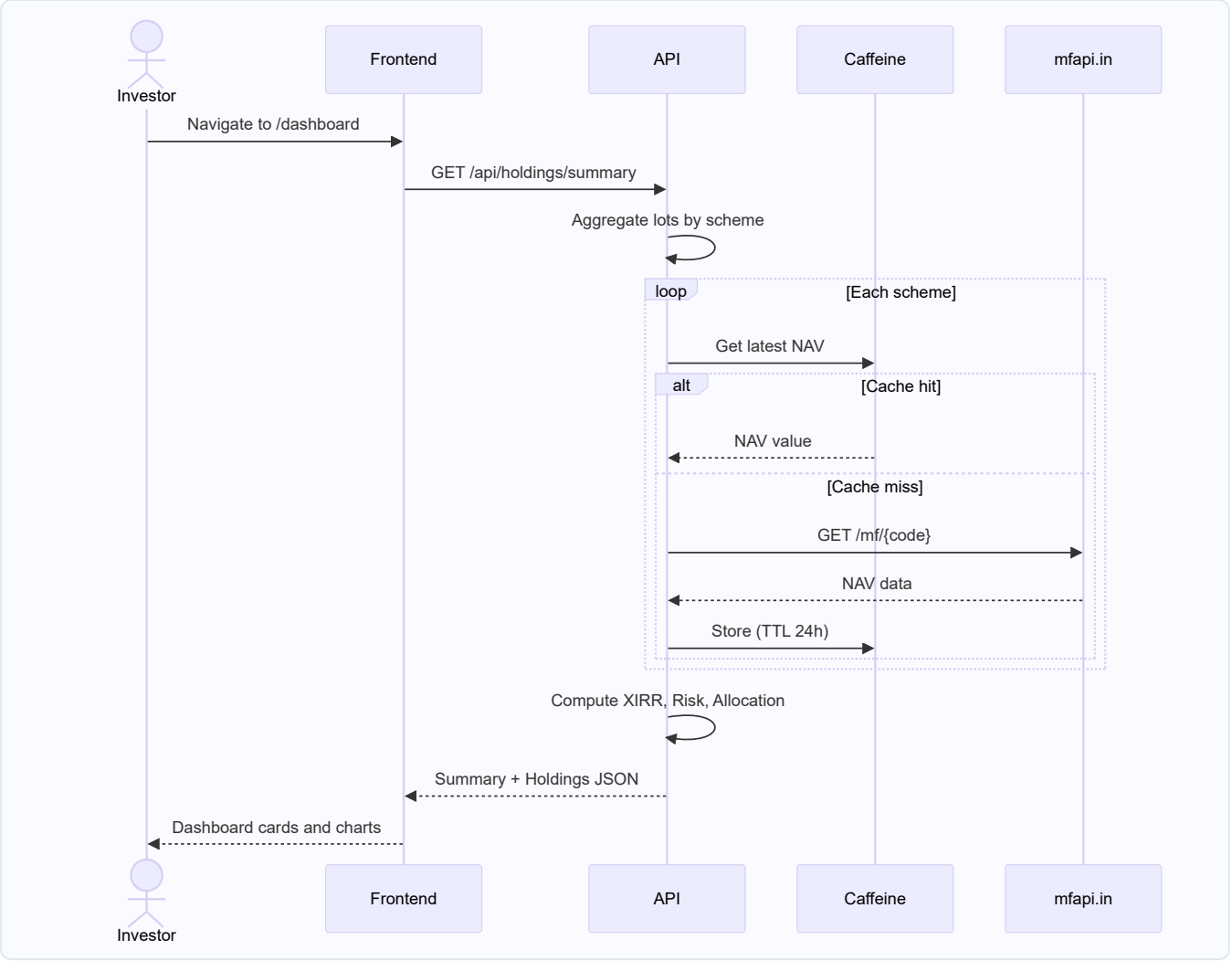
ID	Feature	Priority
FR-01	User Registration (email, password, name)	P0
FR-02	Login with JWT (24h expiry)	P0
FR-03	Password Reset via OTP (BCrypt, 15 min TTL)	P1
FR-04	User Profile Management	P1
FR-05	Notification Preferences CRUD	P2
FR-06	Manual Transaction Entry (11 types)	P0
FR-07	Bulk SIP Generator (up to 120 months)	P1
FR-08	CAS PDF Import (PDFBox + fuzzy match)	P1
FR-09	Investment Plan Management	P1
FR-10	AMFI Scheme Master Seed (45,000+ schemes)	P0
FR-11	NAV Sync — Latest (Caffeine 24h cache)	P0
FR-12	NAV Sync — Historical (Caffeine 7d cache)	P1
FR-13	Scheme Fuzzy Search (GIN trigram index)	P1
FR-14	Per-Fund Holdings Computation	P0
FR-15	Portfolio Summary (XIRR, risk, allocation)	P0
FR-16	Portfolio Growth Timeline (monthly)	P1
FR-17	Risk Profile (Sharpe, volatility, drawdown)	P1
FR-18	SIP Intelligence (streaks, outflow)	P1
FR-19	Financial Goal CRUD	P0
FR-20	Goal-Fund Linking with allocation %	P1
FR-21	Deterministic Goal Projection + Sensitivity	P1
FR-22	Monte Carlo Goal Simulation (10,000 paths)	P1
FR-23	Tax Report — STCG/LTCG (FIFO)	P1
FR-24	User Alerts Management	P1
FR-25	Per-type Notification Preferences	P2

3. Use Cases

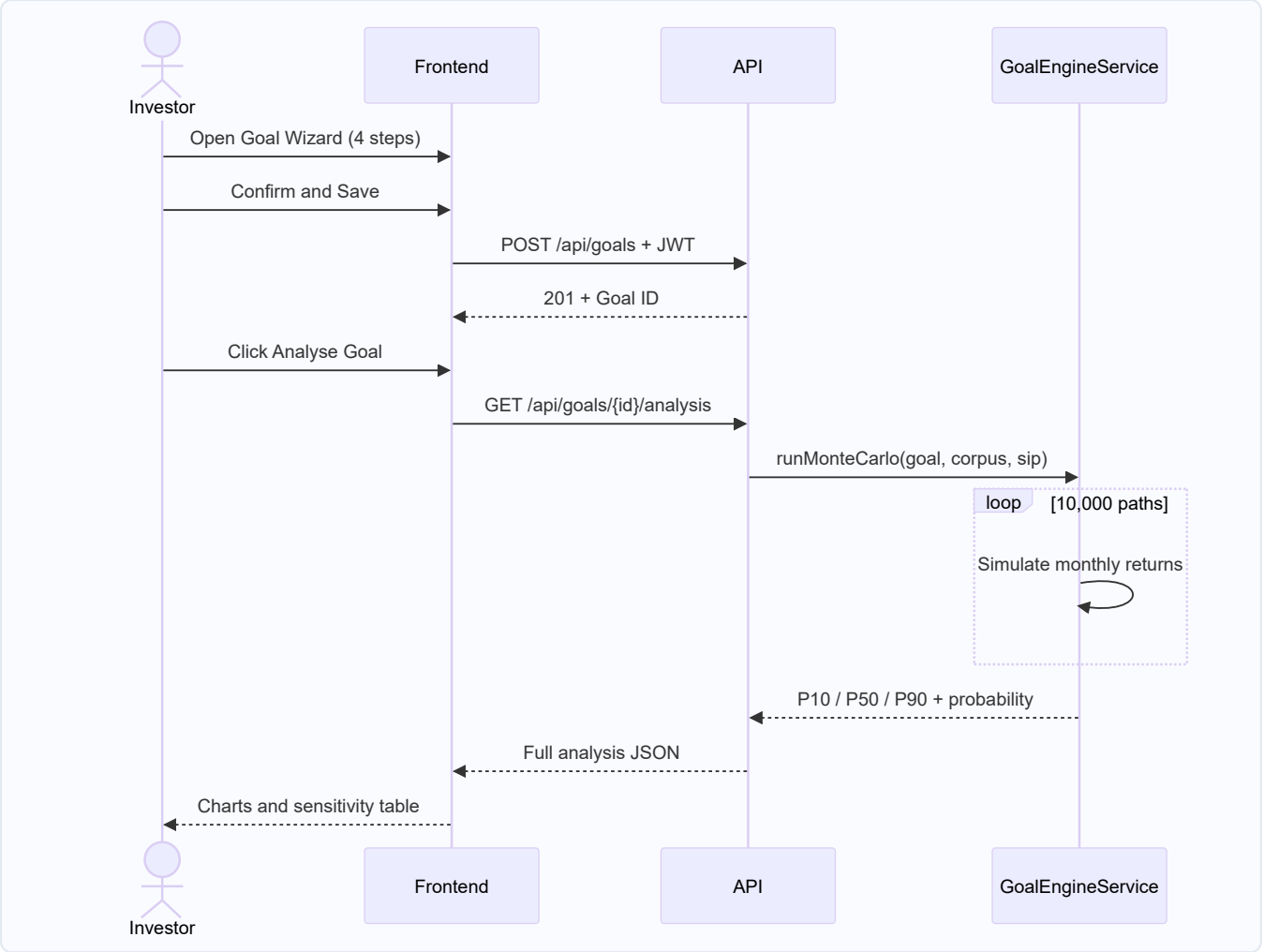
UC-01: Record a Transaction



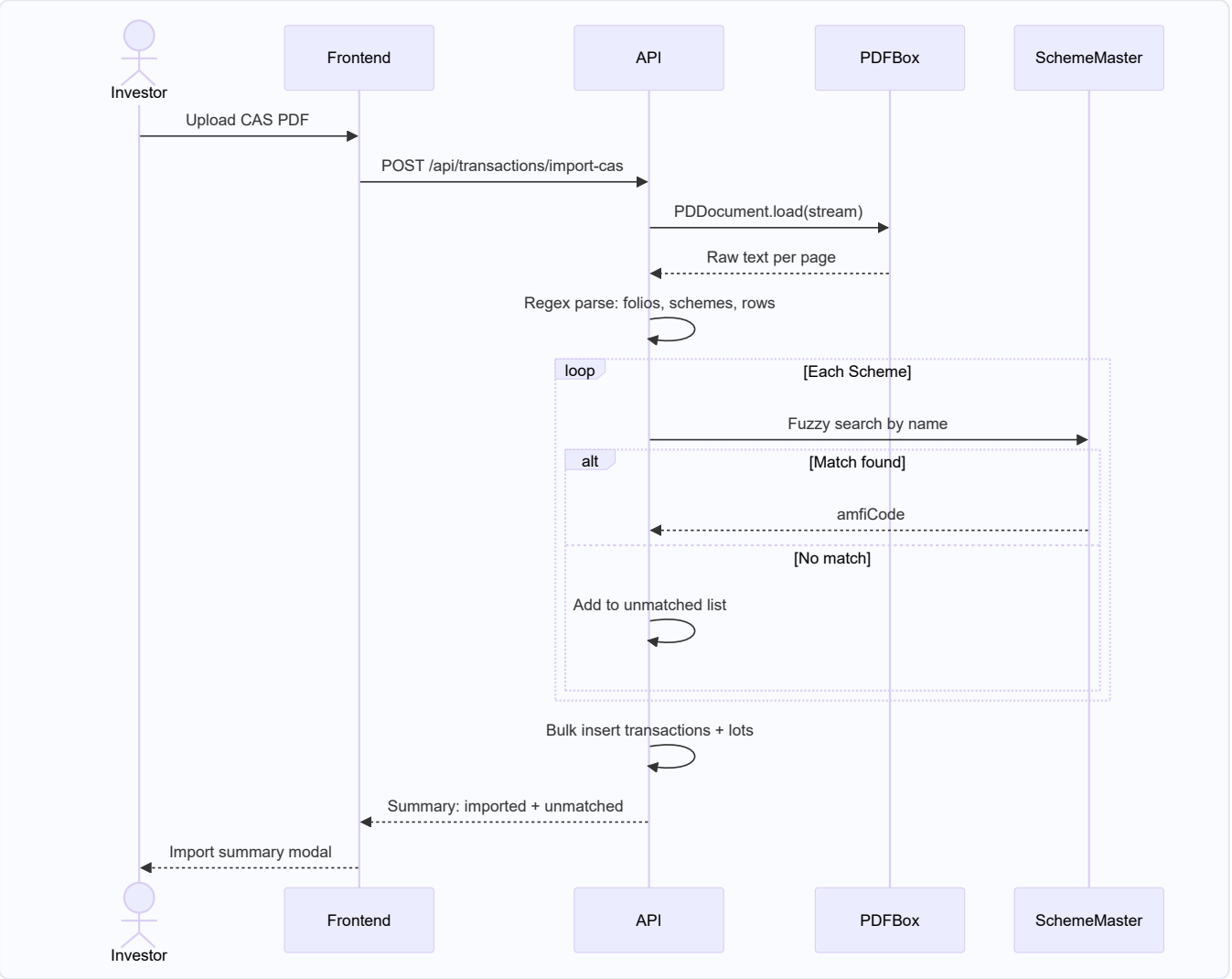
UC-02: View Portfolio Dashboard



UC-03: Create and Analyse a Goal



UC-04: Import CAS PDF



4. Non-Functional Requirements

4.1 Performance Targets

ID	Metric	Target
NFR-01	Dashboard Time to Interactive	< 3 seconds
NFR-02	API P95 response time	< 2,000 ms
NFR-03	NAV cache hit rate	≥ 90%
NFR-04	Concurrent users (512 MB container)	≥ 50
NFR-06	Scheme search response	< 100 ms
NFR-07	CAS PDF parse (200 transactions)	< 30 seconds
NFR-08	Monte Carlo (10,000 iterations)	< 2 seconds

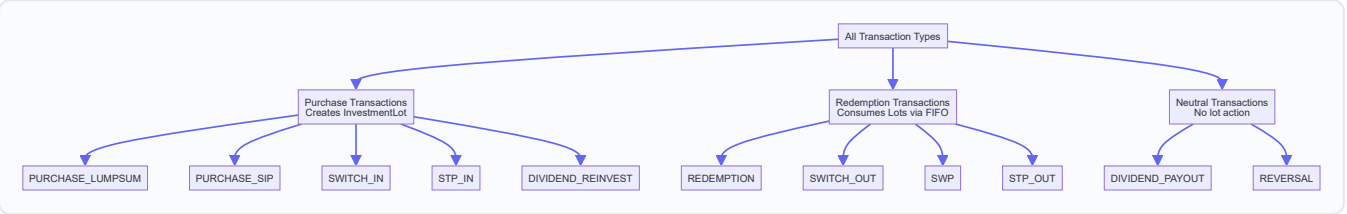
4.2 Security Requirements

ID	Requirement	Standard
NFR-10	Passwords stored as BCrypt hash, cost ≥ 10	OWASP
NFR-11	JWTs HMAC-SHA256 signed, 24h expiry	RFC 7519
NFR-12	PAN cards AES-256-GCM encrypted at rest	SEBI/IT Act
NFR-13	OTPs: SecureRandom, BCrypt hashed, 15 min TTL	OWASP
NFR-14	HSTS, X-Frame-Options: DENY, nosniff header	OWASP
NFR-15	CORS restricted to allow-listed origins	RFC 6454

4.3 Quality Requirements

ID	NFR	Target
NFR-20	Monthly uptime	≥ 99%
NFR-21	Financial data ACID compliance	Full
NFR-22	Responsive design	≥ 375px viewport
NFR-23	Unit test coverage	≥ 70% on core services
NFR-24	Docker image size	≤ 300 MB
NFR-25	Currency formatting	₹ INR, en-IN locale

5. Transaction Type Classification



6. Data Requirements

6.1 Primary Entities

Entity	Primary Key	Key Attributes
`users`	UUID	email, password_hash, pan (encrypted), risk_tolerance
`user_profiles`	UUID	user_id (FK), phone, date_of_birth, currency_format
`investment_plans`	UUID	user_id (FK), fund_name, amount, frequency, goal_id
`investment_transactions`	UUID	user_id (FK), scheme_code, transaction_type, amount, nav, units
`investment_lots`	UUID	user_id (FK), transaction_id (FK), units_purchased, units_remaining
`financial_goals`	UUID	user_id (FK), name, target_amount, target_date, inflation_rate
`goal_fund_links`	UUID	goal_id (FK), scheme_code, allocation_pct
`scheme_master`	BIGSERIAL	amfi_code (UK), scheme_name, amc_name, category, last_nav
`nav_daily`	BIGSERIAL	scheme_code, nav_date, nav_value (UNIQUE: code + date)
`user_alerts`	UUID	user_id (FK), title, severity (Info/Watch/Action), is_read

DOCUMENT 03

System Design: HLD & LLD

WealthWise Documentation Suite · April 2026

WealthWise — System Design: HLD & LLD

Version: 2.0
Date: April 2026

PART I: HIGH-LEVEL DESIGN (HLD)

1. Architecture Overview

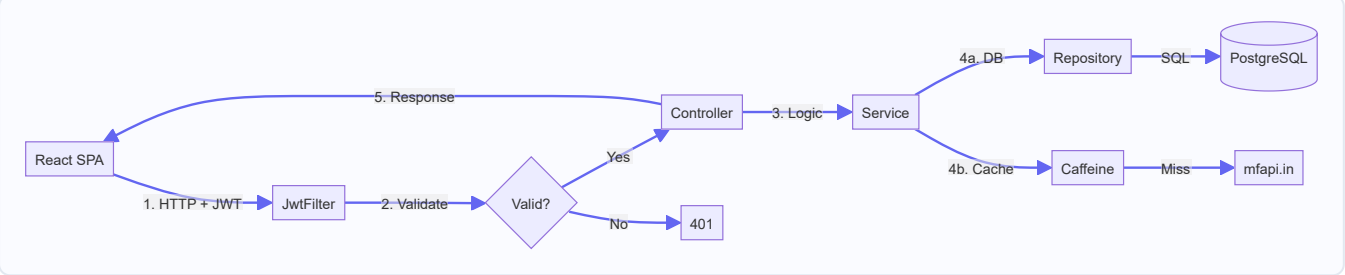
WealthWise follows a **three-tier client-server architecture** with strict separation of concerns. The design prioritizes:

1. **Statelessness** — No server-side sessions; all authentication state carried in JWT tokens
2. **API-First Design** — Backend exposes a pure REST API; frontend is a completely decoupled SPA
3. **Cache-Layered Data Access** — Caffeine in-memory caching reduces external API calls by 95%+
4. **Security-by-Default** — Every response includes hardened security headers; sensitive data encrypted at rest

1.1 Architecture Diagram

Client Tier
React 19 SPA
Vite 8 + React Router 7
HTTPS + JWT
Application Tier
Spring Boot 3.2.3
JWT Filter + BCrypt
Service Layer
Data Tier
PostgreSQL 15
Supabase
Caffeine Cache
Miss
External
mfapi.in
AMFI NAVAll
Gmail SMTP

1.2 Request Processing Pipeline



2. Technology Justification

2.1 Frontend Technology Choices

Choice	Alternative Considered	Justification
React 19	Vue 3, Angular 17	Concurrent rendering features, largest ecosystem, team familiarity
Vite 8	Create React App, webpack	10–100× faster builds, native ESM, minimal config
React Router 7	TanStack Router	Mature API, loaders/actions, protected route support
Recharts	Chart.js, Nivo, Victory	React-native, responsive containers, composable
Framer Motion	CSS animations, GSAP	Declarative, spring physics, best React DX

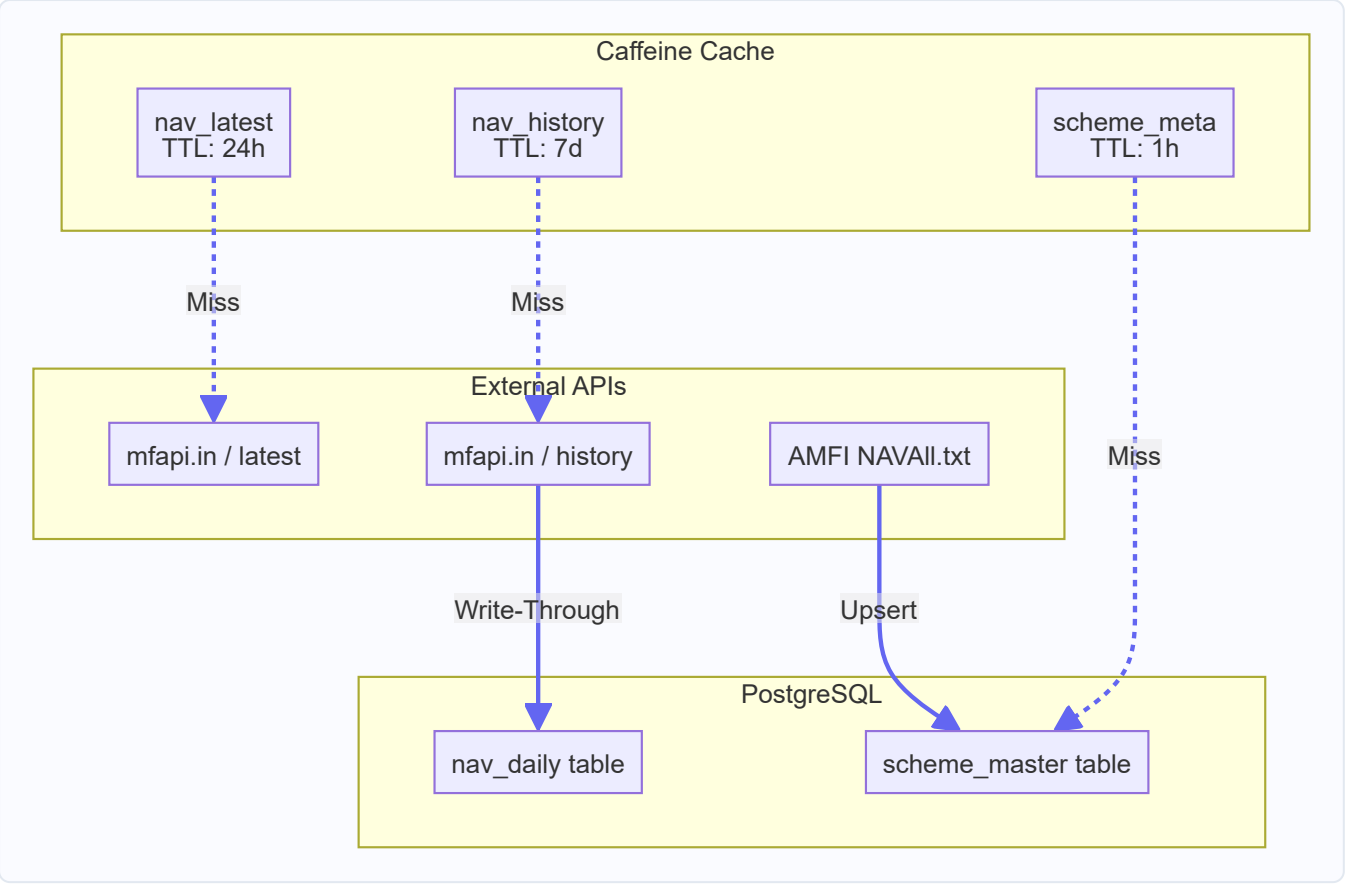
2.2 Backend Technology Choices

Choice	Alternative Considered	Justification
Spring Boot 3.2	Node.js/Express, Django	JVM performance, enterprise-grade security, BigDecimal precision for financials
Spring Security	Custom JWT, Nimbus JOSE	Industry standard, declarative filter chain
Caffeine Cache	Redis, Ehcache	No infrastructure overhead, in-process speed
Apache PDFBox	iText, Tabula	Pure Java, no OS dependencies, free for commercial use
JJWT 0.12.5	Auth0 JWT	Modern type-safe builder API, HMAC-SHA256 signing

2.3 Database Design Choices

Choice	Justification
UUID primary keys	Globally unique, no sequence guess attacks, distributed-safe
NUMERIC(14,2) for money	No floating-point rounding errors; exact decimal arithmetic
GIN trigram index	Sub-100ms fuzzy scheme name search across 45,000+ records
JSONB for audit_log	Schema-flexible old/new value capture without DDL changes
Soft-delete pattern	`is_deleted` flag preserves audit trail

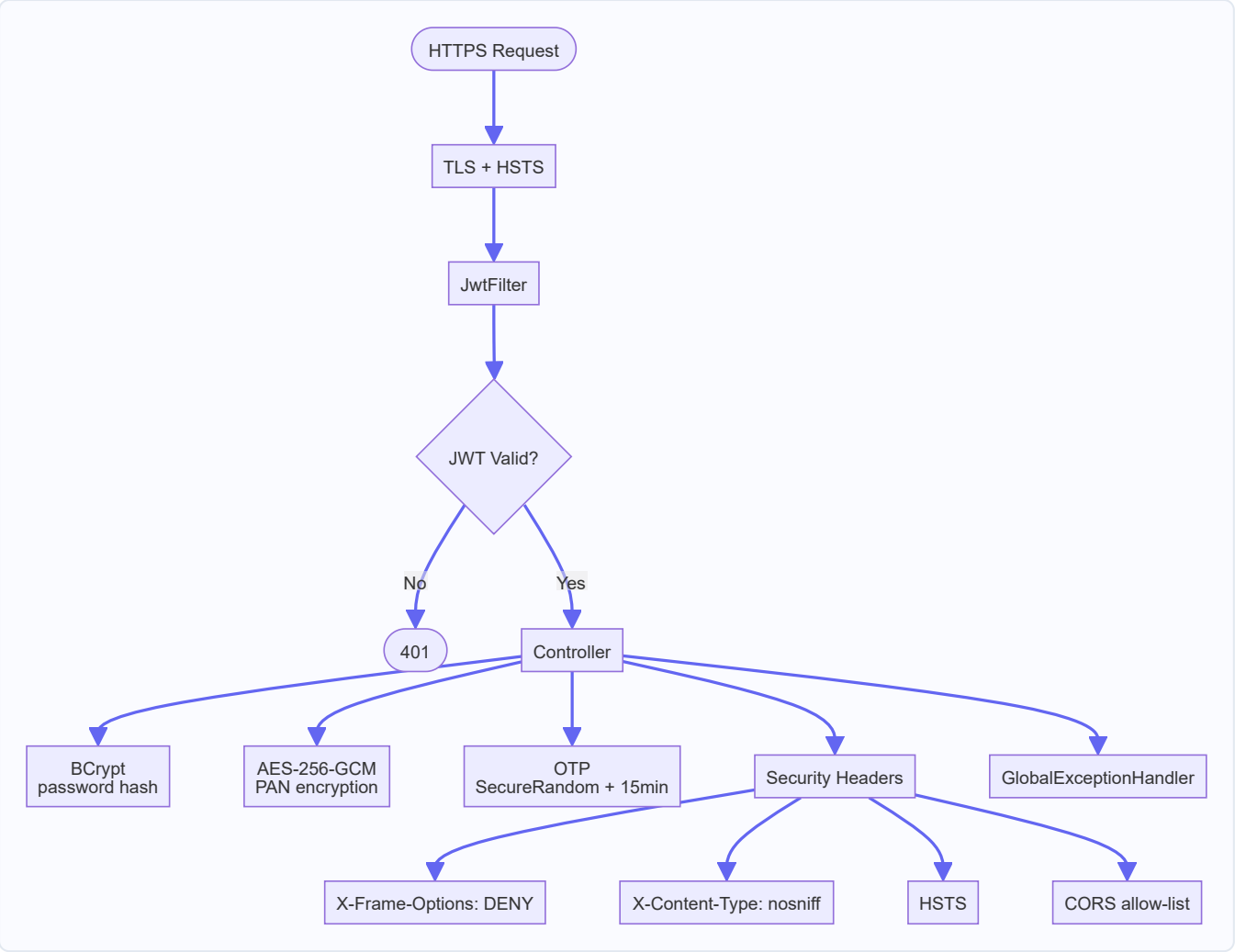
3. Caching Architecture



Design Rationale:

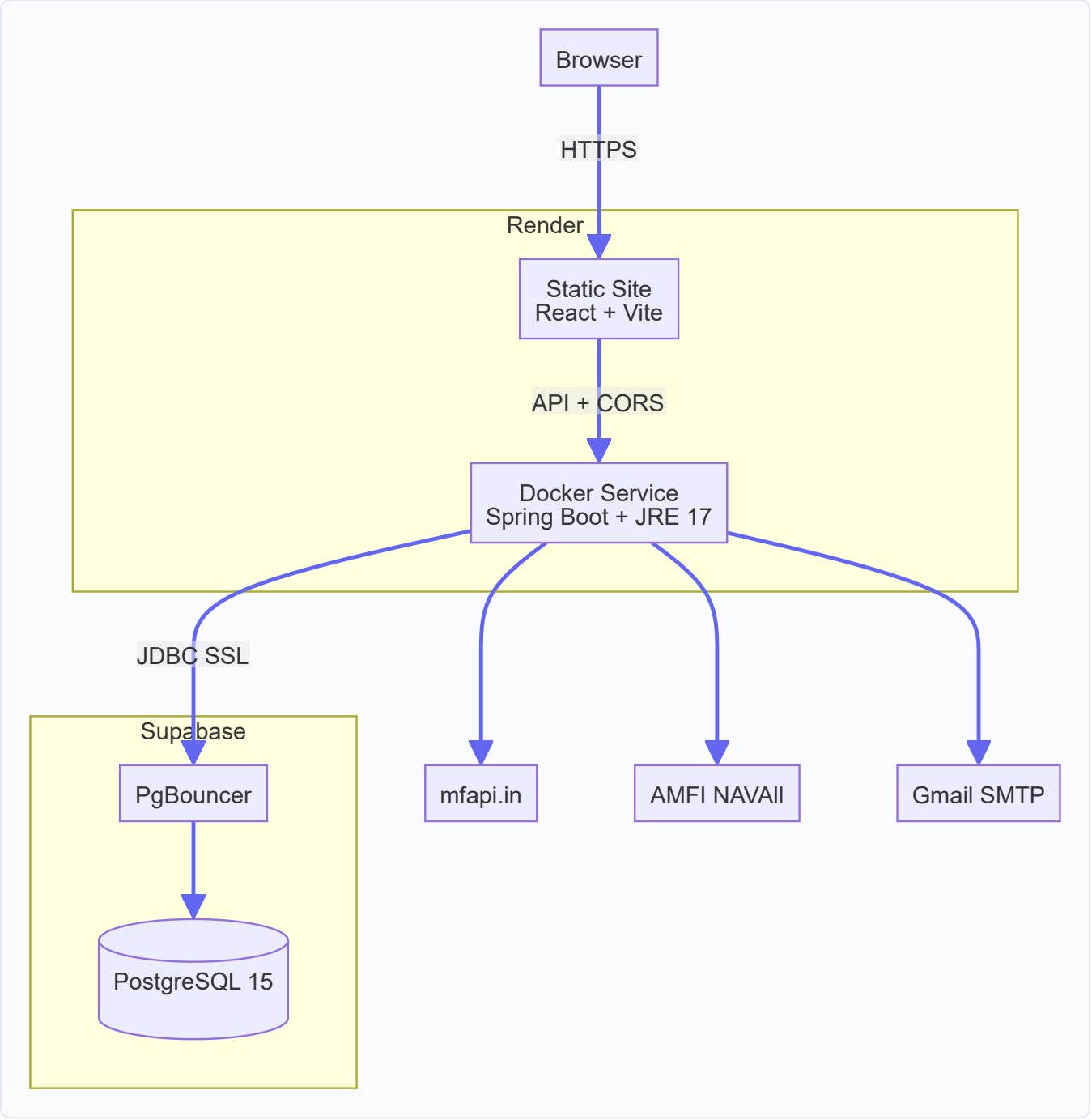
- NAV data is near-immutable once published → 7-day cache is safe for historical NAVs
- Latest NAV updates once daily after AMFI publishes → 24-hour cache is optimal
- Caffeine eliminates 95%+ of external API calls, keeping mfapi.in within rate limits

4. Security Architecture



Security Decision	Rationale
BCrypt over SHA-256	Adaptive cost factor; resistant to rainbow tables and GPU attacks
AES-256-GCM over AES-CBC	Provides authenticated encryption — detects tampering; no padding oracle
Key derived from JWT secret	Avoids introducing a second secret; SHA-256 hash produces strong 256-bit AES key
OTP hashed (not plaintext)	If DB is compromised, attacker cannot use stored OTPs
15-minute OTP expiry	Limits the attack window for intercepted OTPs

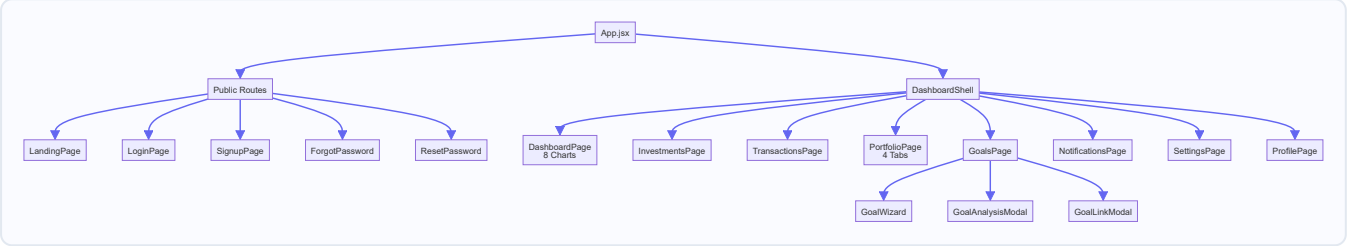
5. Deployment Architecture



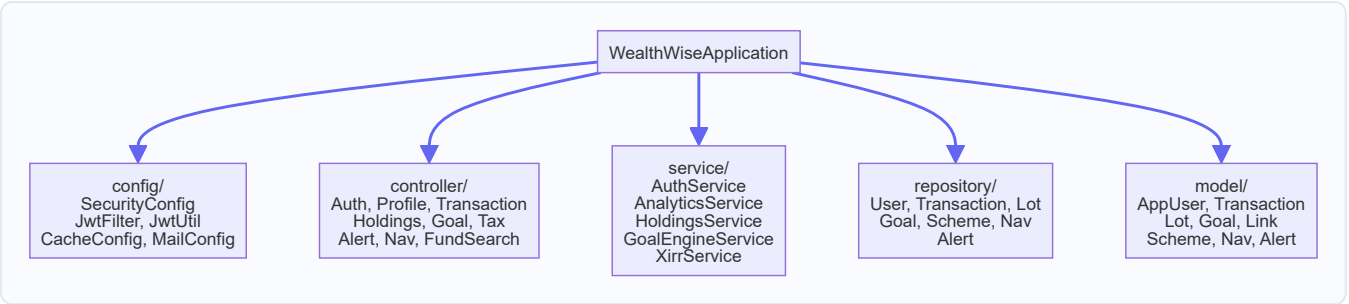
PART II: LOW-LEVEL DESIGN (LLD)

6. Component Architecture

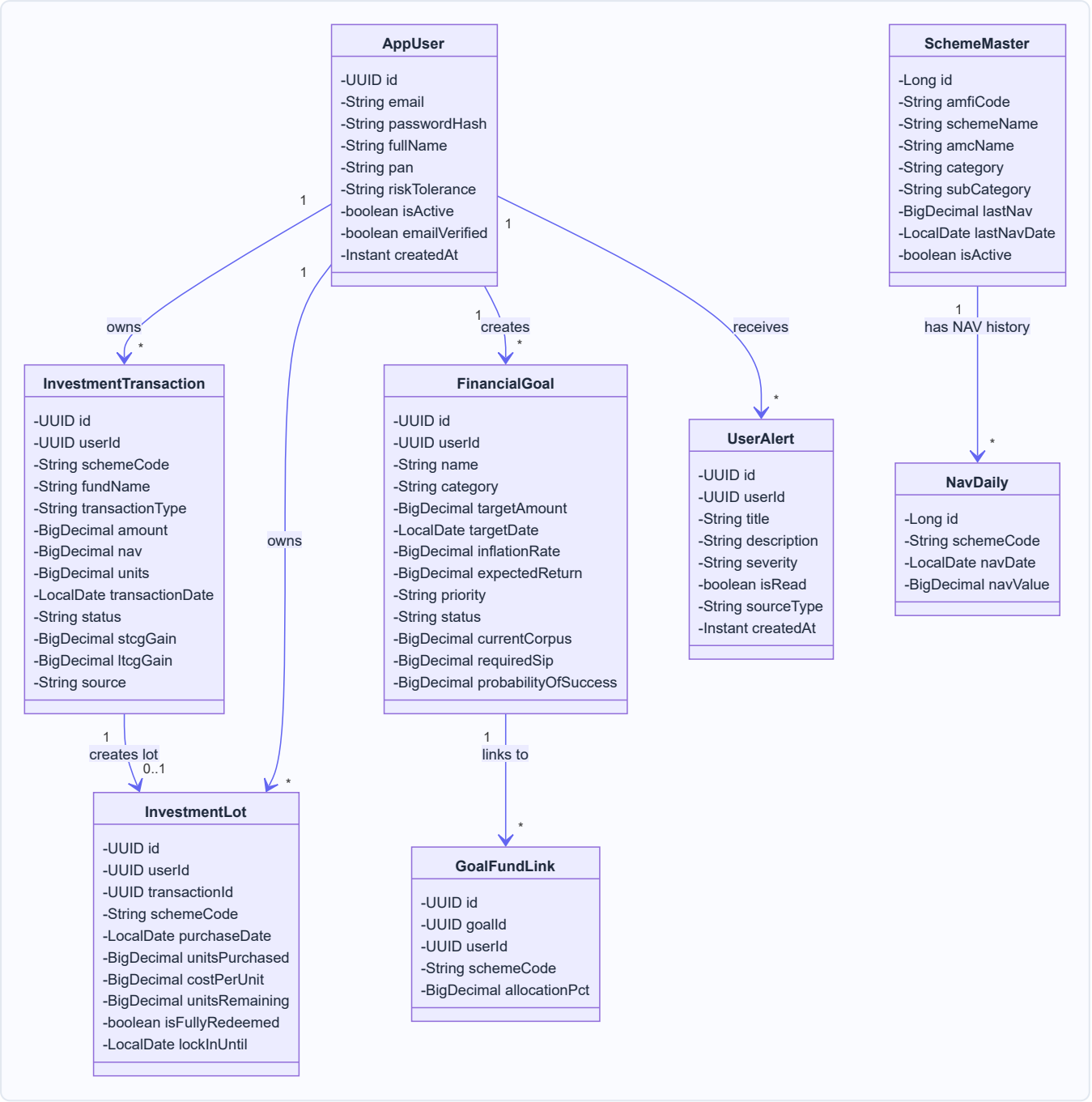
6.1 Frontend Component Tree



6.2 Backend Package Structure

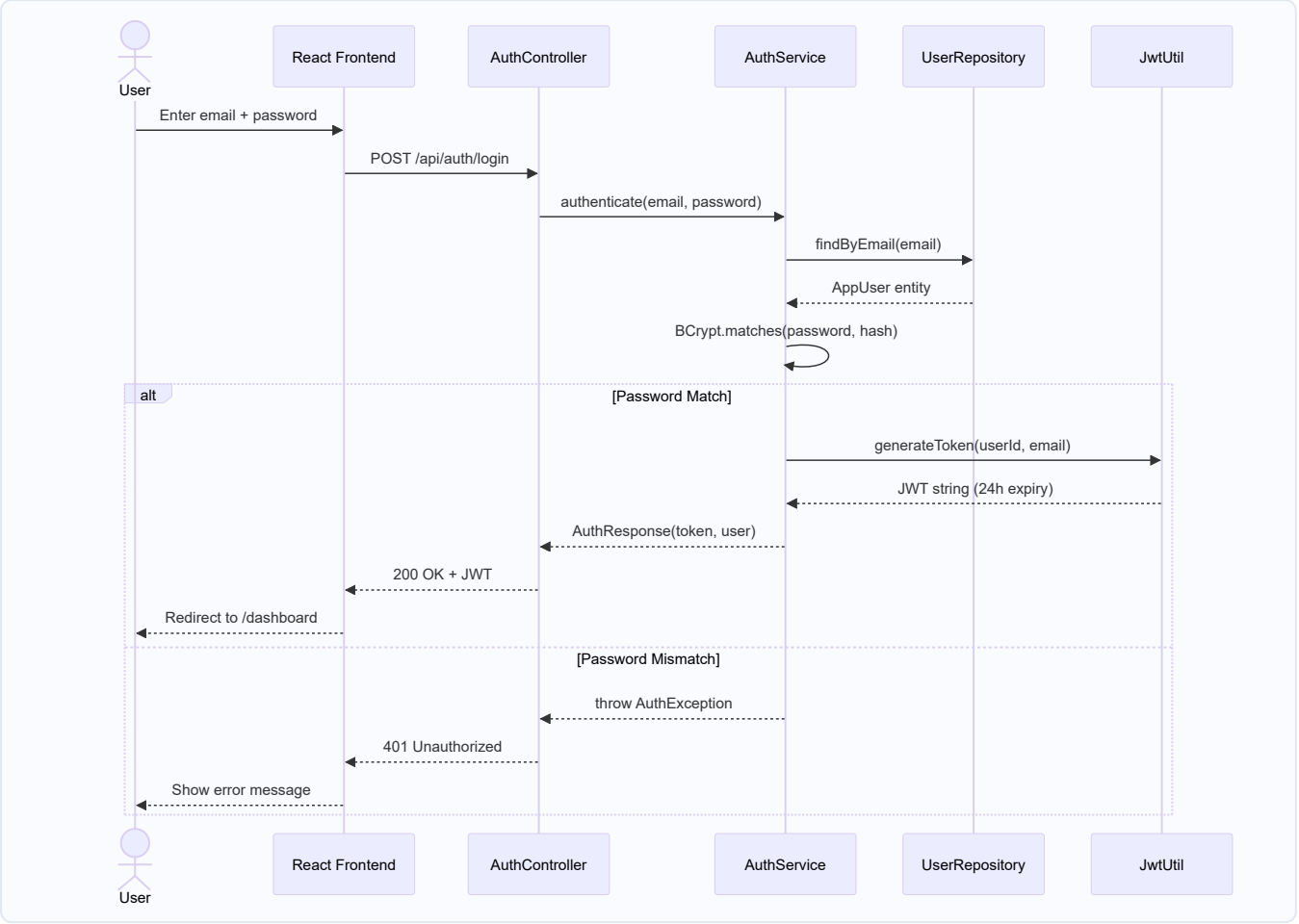


7. Class Diagram

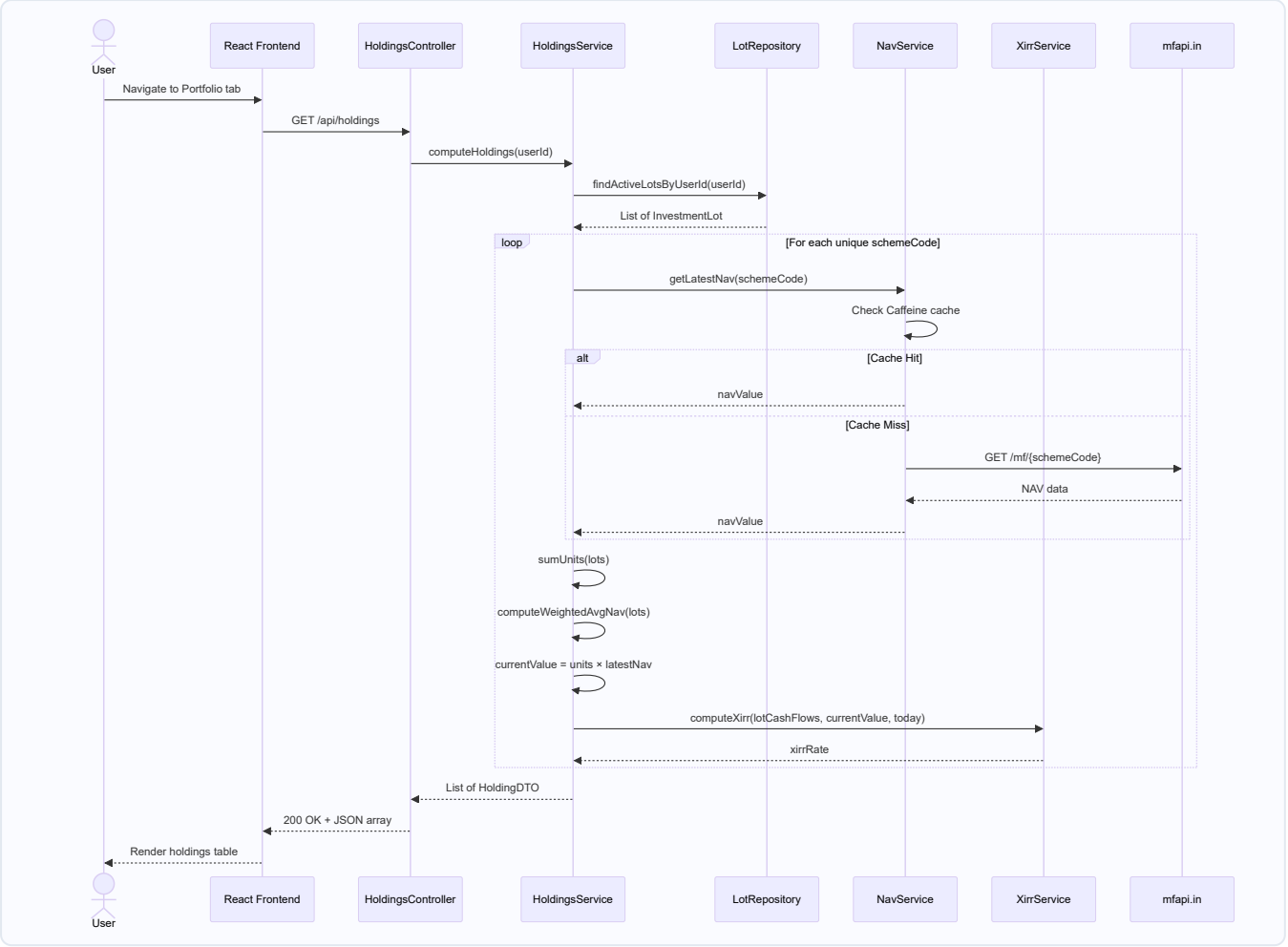


8. Sequence Diagrams

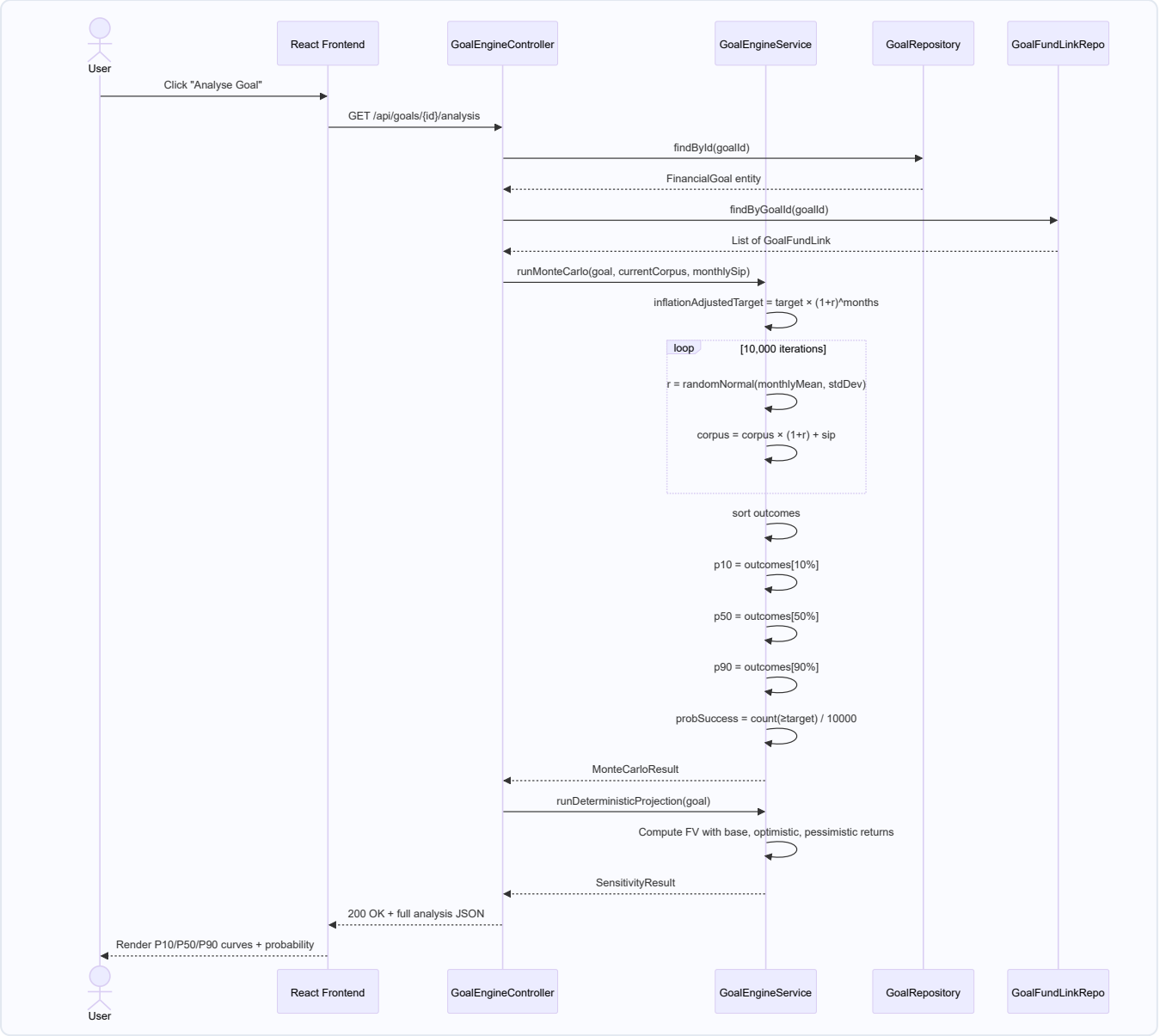
8.1 User Login Flow



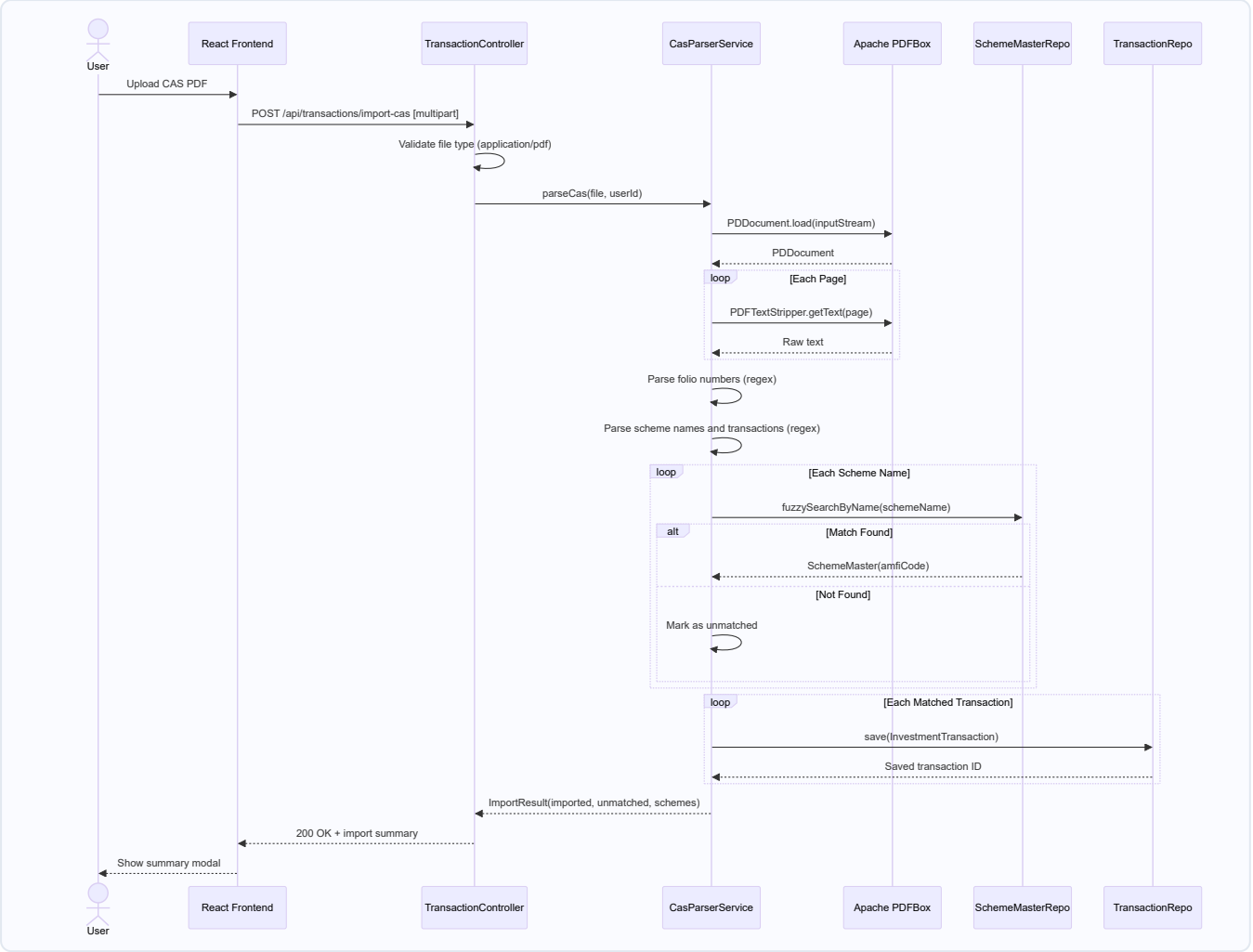
8.2 Portfolio Holdings Computation Flow



8.3 Monte Carlo Goal Analysis Flow

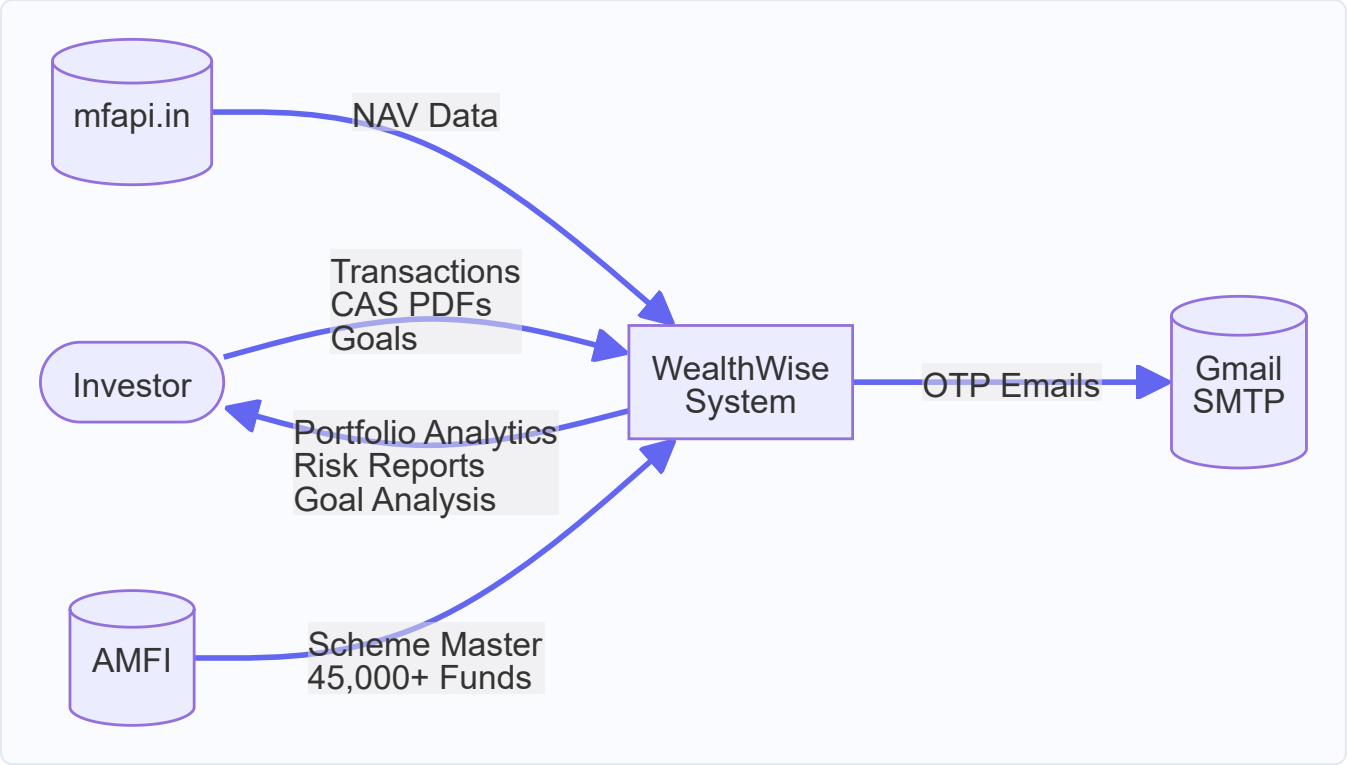


8.4 CAS PDF Import Flow

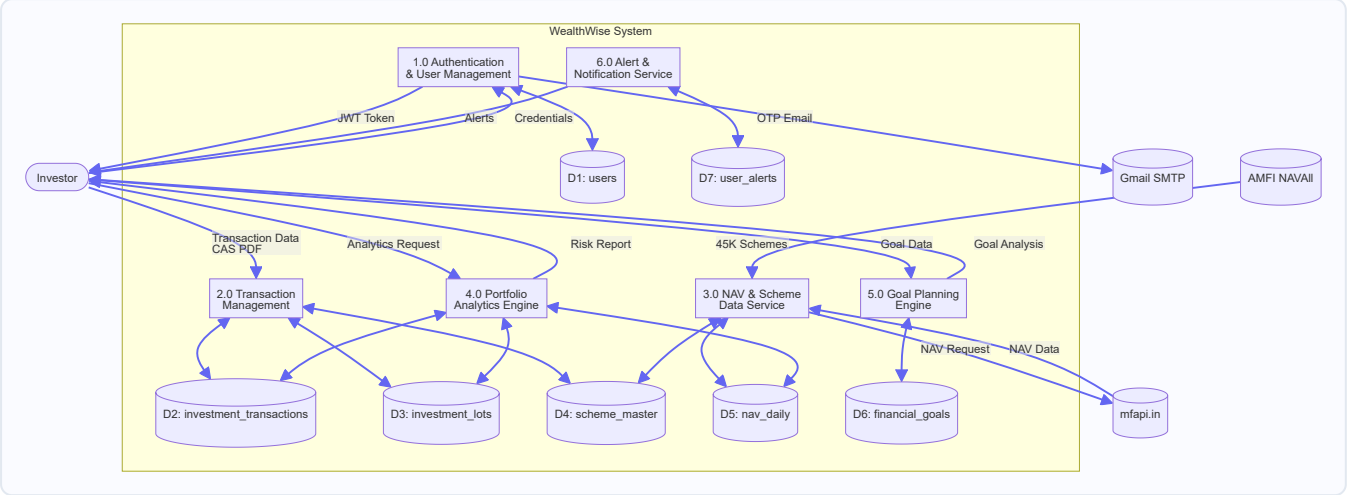


9. Data Flow Diagrams

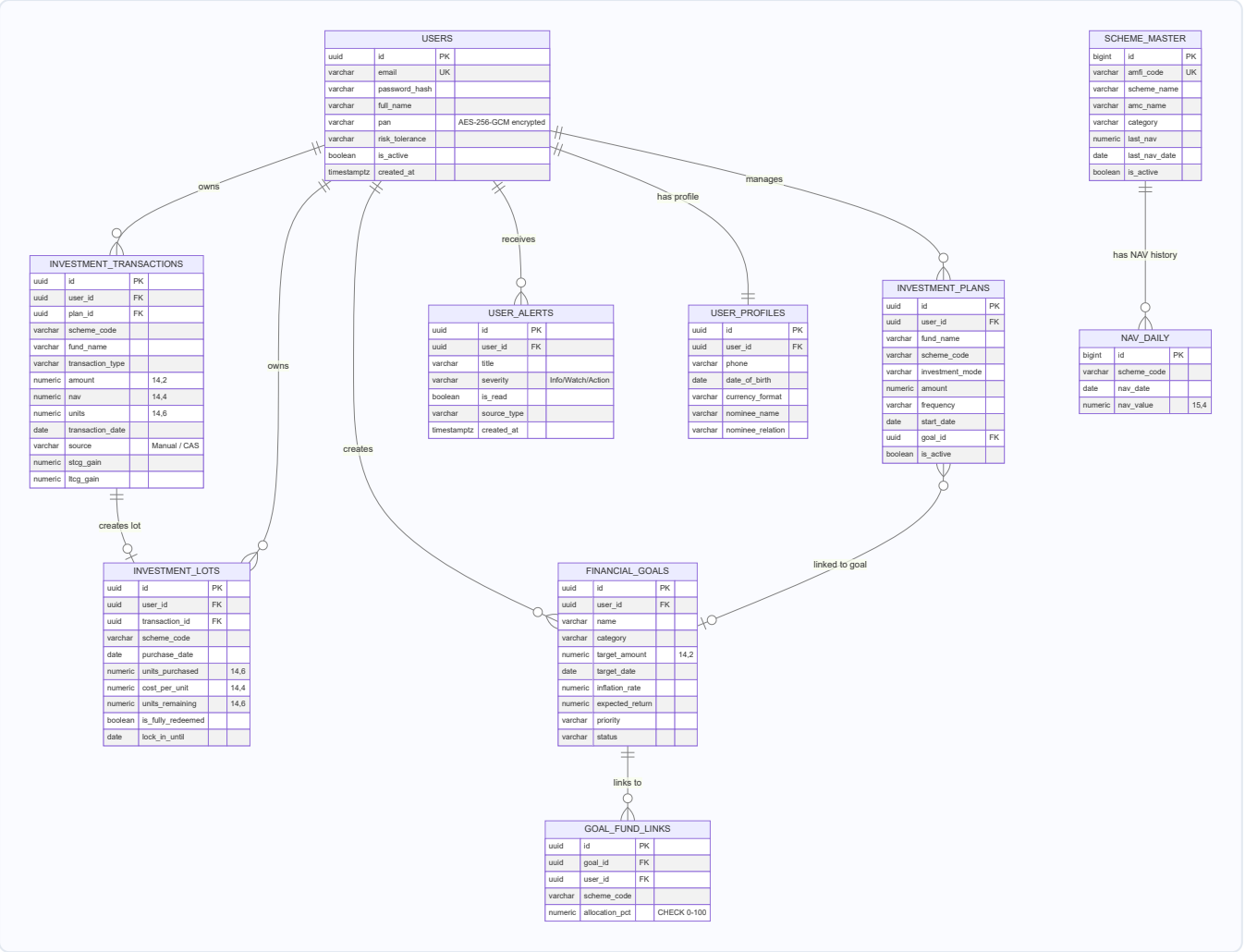
9.1 DFD Level 0 — Context Diagram



9.2 DFD Level 1 — Major Processes



10. Entity-Relationship Diagram


USERS
uuid id PK
varchar email UK
varchar password_hash
varchar full_name
varchar pan AES-256-GCM encrypted
varchar risk_tolerance
boolean is_active
timestamptz created_at
SCHEME_MASTER
bigint id PK
varchar amfi_code UK
varchar scheme_name
varchar amc_name
varchar category
numeric last_nav
date last_nav_date
boolean is_active
owns
manages
has profile
receives
has NAV history
INVESTMENT_TRANSACTIONS
uuid id PK
uuid user_id FK
uuid plan_id FK
varchar scheme_code
varchar fund_name
varchar transaction_type
numeric amount 14,2
numeric nav 14,4
numeric units 14,6
date transaction_date
varchar source Manual / CAS
numeric stcg_gain
numeric ltcg_gain
USER_ALERTS
uuid id PK
uuid user_id FK
varchar title
varchar severity Info/Watch/Action
boolean is_read
varchar source_type
timestamptz created_at
USER_PROFILES
uuid id PK
uuid user_id FK
varchar phone
date date_of_birth
varchar currency_format
varchar nominee_name
varchar nominee_relation
INVESTMENT_PLANS
uuid id PK
uuid user_id FK
varchar fund_name
varchar scheme_code
varchar investment_mode
numeric amount
varchar frequency
date start_date
uuid goal_id FK
boolean is_active
NAV_DAILY
bigint id PK
varchar scheme_code
date nav_date
numeric nav_value 15,4
owns
creates
creates lot
linked to goal
INVESTMENT_LOTS
uuid id PK
uuid user_id FK
uuid transaction_id FK
varchar scheme_code
date purchase_date
numeric units_purchased 14,6
numeric cost_per_unit 14,4
numeric units_remaining 14,6
boolean is_fully_redeemed
date lock_in_until
FINANCIAL_GOALS
uuid id PK
uuid user_id FK
varchar name
varchar category
numeric target_amount 14,2
date target_date
numeric inflation_rate
numeric expected_return
varchar priority
varchar status
links to
GOAL_FUND_LINKS
uuid id PK
uuid goal_id FK
uuid user_id FK
varchar scheme_code
numeric allocation_pct CHECK 0-100

11. Algorithm Pseudocode

11.1 XIRR — Newton-Raphson Method

```
FUNCTION computeXIRR(cashFlows: List<{date, amount}>) → Double:
    // cashFlows: purchase lots as negative values,
    //            current market value as positive terminal value (today)

    rate ← 0.1   // initial guess: 10% annual return

    FOR iteration = 1 TO 100:
        npv ← 0
        dnpv ← 0
        baseDate ← cashFlows[0].date

        FOR EACH (date, amount) IN cashFlows:
            t ← daysBetween(baseDate, date) / 365.25
            npv  ← npv  + amount / (1 + rate)^t
            dnpv ← dnpv - t × amount / (1 + rate)^(t + 1)

        IF |dnpv| < 1e-10: BREAK   // derivative too small

        newRate ← rate - npv / dnpv

        IF |newRate - rate| < 1e-7: RETURN newRate   // converged

        rate ← newRate

    RETURN rate   // best approximation after 100 iterations
```

11.2 FIFO Lot Disposal (Redemption)

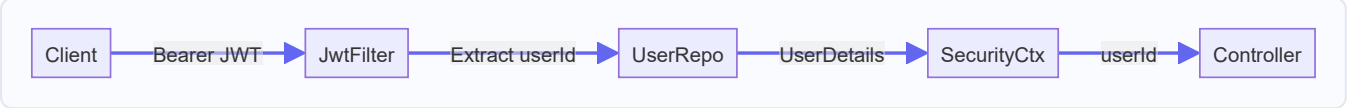
```
FUNCTION processRedemption(userId, schemeCode, redeemUnits, redeemNav, date):
    lots ← LotRepo.findByUserAndSchemeOrderByPurchaseDateAsc(userId, schemeCode)
    remaining ← redeemUnits
    stcgGain ← 0
    ltcgGain ← 0

    FOR lot IN lots:
        IF remaining ≤ 0: BREAK
        IF lot.lockInUntil ≠ null AND date < lot.lockInUntil:
            THROW LockInException(lot.schemeCode, lot.lockInUntil)

        consume ← MIN(lot.unitsRemaining, remaining)
        gain ← consume × (redeemNav - lot.costPerUnit)
        holdingDays ← daysBetween(lot.purchaseDate, date)

        IF holdingDays ≥ 365:
            ltcgGain ← ltcgGain + gain
        ELSE:
            stcgGain ← stcgGain + gain

        lot.unitsRemaining ← lot.unitsRemaining - consume
        IF lot.unitsRemaining ≤ 0:
            lot.isFullyRedeemed ← true

        remaining ← remaining - consume

    IF remaining > 0:
        THROW InsufficientUnitsException

    SAVE updated lots
    RETURN {stcgGain, ltcgGain}
```

11.3 Monte Carlo Goal Simulation

```
FUNCTION runMonteCarlo(goal, currentCorpus, monthlySip, iterations=10000):
    monthsToTarget ← monthsBetween(NOW, goal.targetDate)
    monthlyMean    ← goal.expectedReturn / 100 / 12
    monthlyStdDev  ← 0.06   // 6% monthly volatility assumption

    inflationAdjTarget ← goal.targetAmount
        × (1 + goal.inflationRate/100)^(monthsToTarget / 12)

    outcomes ← []

    FOR i IN 1..iterations:
        corpus ← currentCorpus

        FOR month IN 1..monthsToTarget:
            r ← randomNormal(monthlyMean, monthlyStdDev)
            corpus ← corpus × (1 + r) + monthlySip

        outcomes.append(corpus)

    outcomes.sort()

    p10 ← outcomes[iterations × 0.10]
    p50 ← outcomes[iterations × 0.50]
    p90 ← outcomes[iterations × 0.90]
    probSuccess ← COUNT(outcomes ≥ inflationAdjTarget) / iterations

    RETURN {p10, p50, p90, probSuccess, inflationAdjTarget}
```

12. API Design Patterns

12.1 Authentication Pattern



12.2 Error Response Pattern

All errors return a structured JSON body — no stack traces exposed in API responses:

```
{
  "error": "Human-readable error message",
  "status": 400,
  "timestamp": "2026-04-21T18:00:00Z",
  "path": "/api/transactions"
}
```

Status	Condition
400 Bad Request	Validation failure, malformed body
401 Unauthorized	Missing, expired, or invalid JWT
403 Forbidden	Valid JWT but insufficient permissions
404 Not Found	Resource not found or doesn't belong to user
409 Conflict	Duplicate resource (e.g., email already registered)
500 Internal Server Error	Unhandled exception (sanitized message only)

DOCUMENT 04

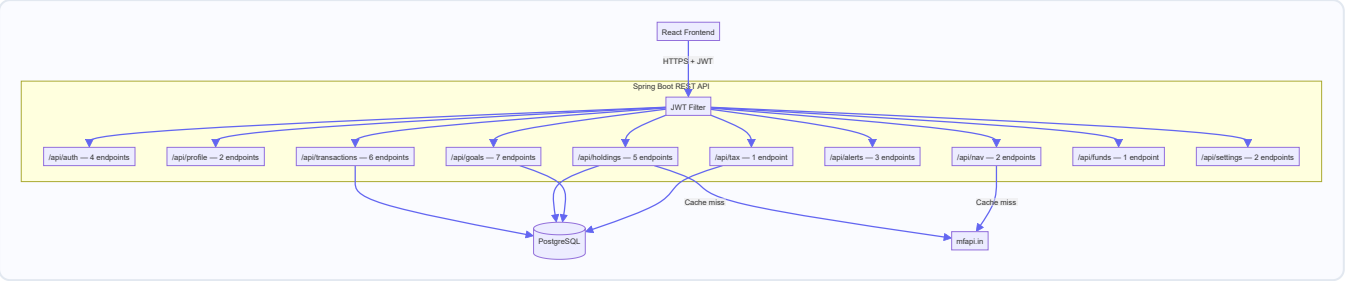
API Documentation

WealthWise Documentation Suite · April 2026

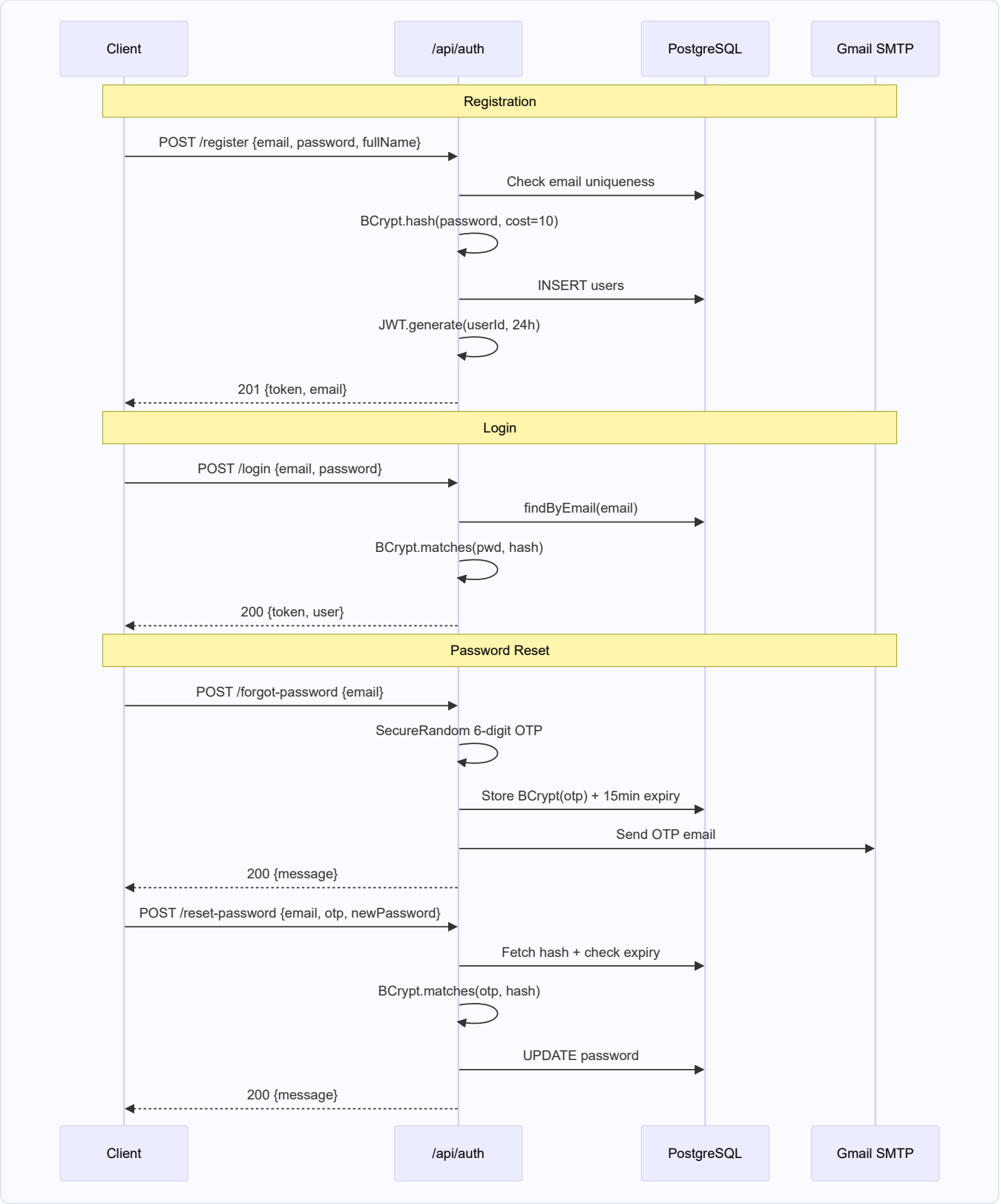
WealthWise — API Documentation

Version: 2.0 | **Date:** April 2026
Base URL (Production): `https://wealthwise-backend.onrender.com`
Base URL (Local): `http://localhost:8080`
Authentication: `Authorization: Bearer <JWT>`
Content-Type: `application/json`

1. API Architecture



2. Authentication Flow



3. Group 1: Authentication (`/api/auth`)

All auth endpoints are **public** (no JWT required).

POST `/api/auth/register`

```
Request:  { "email": "faishal@example.com", "password": "SecurePass@2026", "fullName": "Faishal Rahman Ansari" }
Response: { "token": "eyJhbGci...", "id": "550e8400-...", "email": "faishal@example.com" }
```

POST `/api/auth/login`

```
Request:  { "email": "faishal@example.com", "password": "SecurePass@2026" }
Response: { "token": "eyJhbGci...", "id": "...", "email": "...", "fullName": "..." }
Error:    401 { "error": "Invalid email or password" }
```

POST `/api/auth/forgot-password`

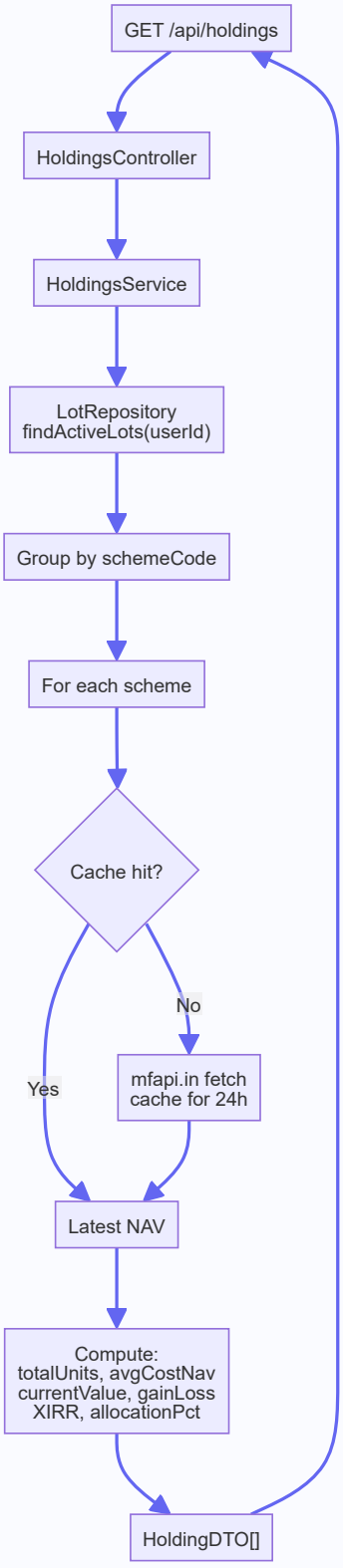
```
Request:  { "email": "faishal@example.com" }
Response: { "message": "OTP sent to your registered email" }
```

POST `/api/auth/reset-password`

```
Request:  { "email": "...", "otp": "483921", "newPassword": "NewPass@2026" }
Response: { "message": "Password reset successful" }
```

---

4. Portfolio Holdings Computation



5. Group 2: Holdings Analytics (`/api/holdings`)

GET `/api/holdings`

Returns per-fund breakdown with XIRR for each holding.

```
[
  {
    "schemeCode": "120503",
    "fundName": "Axis Bluechip Fund - Growth",
    "category": "Equity",
    "totalUnits": 266.489,
    "avgCostNav": 54.21,
    "currentNav": 58.94,
    "investedValue": 14450.00,
    "currentValue": 15704.23,
    "gainLoss": 1254.23,
    "gainLossPct": 8.68,
    "xirr": 12.4,
    "allocationPct": 21.3
  }
]
```

GET `/api/holdings/summary`

```
{
  "totalInvested": 73700.00,
  "currentValue": 86420.50,
  "gainLoss": 12720.50,
  "gainLossPct": 17.26,
  "portfolioXirr": 16.8,
  "portfolioRiskScore": 3.4,
  "riskLabel": "Moderate",
  "holdingYears": 2.1
}
```

GET `/api/holdings/growth`

```
[{ "month": "2025-01-01", "invested": 60000.00, "value": 61200.00, "gain": 1200.00 }]
```

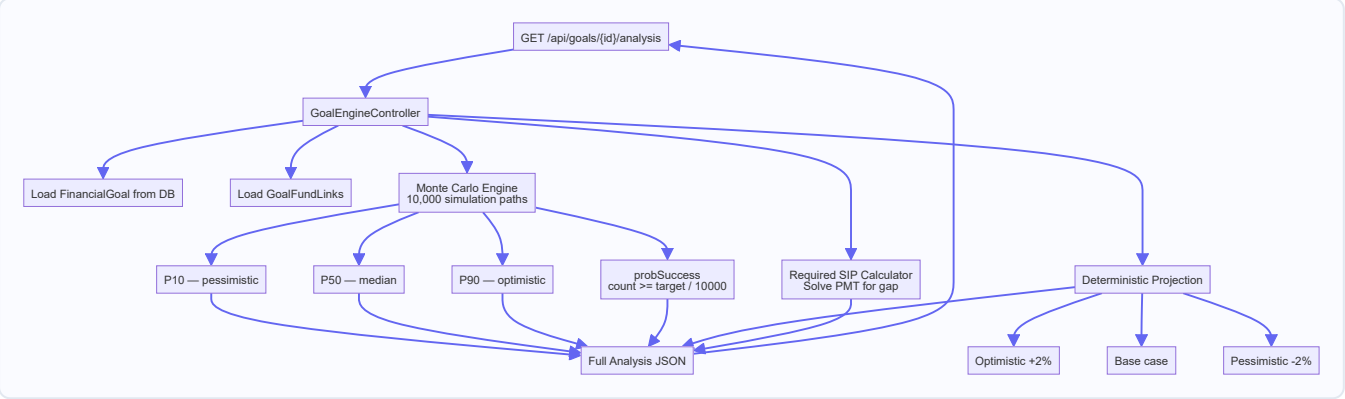
GET `/api/holdings/risk-profile`

```
{ "sharpeRatio": 1.24, "volatility": 14.6, "maxDrawdown": 18.2, "annualReturn": 16.4, "diversificationScore": 7 }
```

GET `/api/holdings/sip-intelligence`

```
[{ "schemeCode": "120503", "fundName": "Axis Bluechip Fund", "monthlySipAmount": 5000.00, "sipStreak": 12 }]
```

6. Goal Analysis Flow



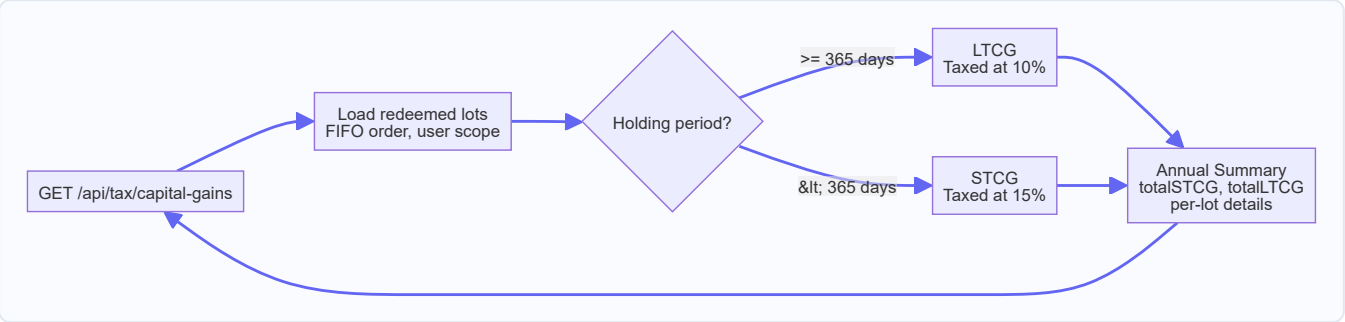
POST `/api/goals` — Create goal

```
Request: { "name": "Retirement", "category": "RETIREMENT", "targetAmount": 10000000, "targetDate": "2045-04-01", "i
nflationRate": 6.0, "expectedReturn": 12.0 }
Response: { "id": "...", "requiredMonthlySip": 18420.00, "probabilityOfSuccess": 0.73 }
```

GET `/api/goals/{id}/analysis`

```
{
  "currentCorpus": 45000.00,
  "inflationAdjustedTarget": 28500000.00,
  "requiredMonthlySip": 18420.00,
  "probabilityOfSuccess": 0.73,
  "monteCarlo": { "p10": 18000000, "p50": 22500000, "p90": 28000000 },
  "sensitivity": {
    "optimistic": { "return": 14.0, "projectedValue": 29000000 },
    "base":       { "return": 12.0, "projectedValue": 22100000 },
    "pessimistic": { "return": 10.0, "projectedValue": 16800000 }
  }
}
```

7. Tax Computation Flow



GET `/api/tax/capital-gains?financialYear=2025-26`

```
{
  "financialYear": "2025-26",
  "totalInvested": 73700.00,
  "totalRedeemed": 12000.00,
  "stcg": 420.00,
  "ltcg": 1650.00,
  "lots": [
    { "fundName": "Axis Bluechip Fund", "purchaseDate": "2024-04-01", "redemptionDate": "2025-06-15",
      "units": 50.0, "costBasis": 2836.00, "redemptionProceeds": 3100.00, "gain": 264.00, "type": "STCG" }
  ]
}
```

8. Complete Endpoint Reference

Public Endpoints (No JWT)

Method	Endpoint	Description
POST	`/api/auth/register`	Register new user
POST	`/api/auth/login`	Login, receive JWT
POST	`/api/auth/forgot-password`	Request OTP
POST	`/api/auth/reset-password`	Reset with OTP

Protected Endpoints (JWT Required)

#	Method	Endpoint	Description
1	GET	`/api/profile`	Get profile
2	PUT	`/api/profile`	Update profile
3	POST	`/api/transactions`	Create transaction
4	GET	`/api/transactions`	List transactions
5	PUT	`/api/transactions/{id}`	Update transaction
6	DELETE	`/api/transactions/{id}`	Delete transaction
7	POST	`/api/transactions/import-cas`	Import CAS PDF
8	GET	`/api/transactions/lots`	List lots
9	POST	`/api/investments`	Create plan
10	GET	`/api/investments`	List plans
11	PUT	`/api/investments/{id}`	Update plan
12	DELETE	`/api/investments/{id}`	Delete plan
13	GET	`/api/holdings`	Per-fund holdings
14	GET	`/api/holdings/summary`	Portfolio totals
15	GET	`/api/holdings/growth`	Monthly timeline
16	GET	`/api/holdings/risk-profile`	Risk analytics
17	GET	`/api/holdings/sip-intelligence`	SIP analytics
18	POST	`/api/goals`	Create goal
19	GET	`/api/goals`	List goals
20	GET	`/api/goals/{id}`	Get goal
21	PUT	`/api/goals/{id}`	Update goal
22	DELETE	`/api/goals/{id}`	Delete goal
23	GET	`/api/goals/{id}/analysis`	Monte Carlo analysis
24	POST	`/api/goals/{id}/links`	Link fund to goal
25	DELETE	`/api/goals/{goalId}/links/{linkId}`	Unlink fund
26	GET	`/api/tax/capital-gains`	STCG/LTCG report
27	GET	`/api/alerts`	List alerts
28	PATCH	`/api/alerts/{id}/read`	Mark read
29	POST	`/api/alerts/mark-all-read`	Mark all read
30	GET	`/api/nav/latest/{code}`	Latest NAV
31	GET	`/api/nav/history/{code}`	NAV history
32	GET	`/api/funds/search?q=`	Fuzzy fund search
33	GET	`/api/settings/notifications`	Get preferences
34	PUT	`/api/settings/notifications`	Update preferences

9. Error Response Format

All errors return a consistent JSON structure — no stack traces exposed:

```
{
  "error": "Human-readable error message",
  "status": 400,
  "timestamp": "2026-04-21T18:00:00Z",
  "path": "/api/transactions"
}
```

Status	Condition
400	Validation failure, malformed body
401	Missing, expired, or invalid JWT
403	Valid JWT but insufficient permissions
404	Resource not found
409	Duplicate resource (e.g., email exists)
500	Unhandled exception (sanitized message)

DOCUMENT 05

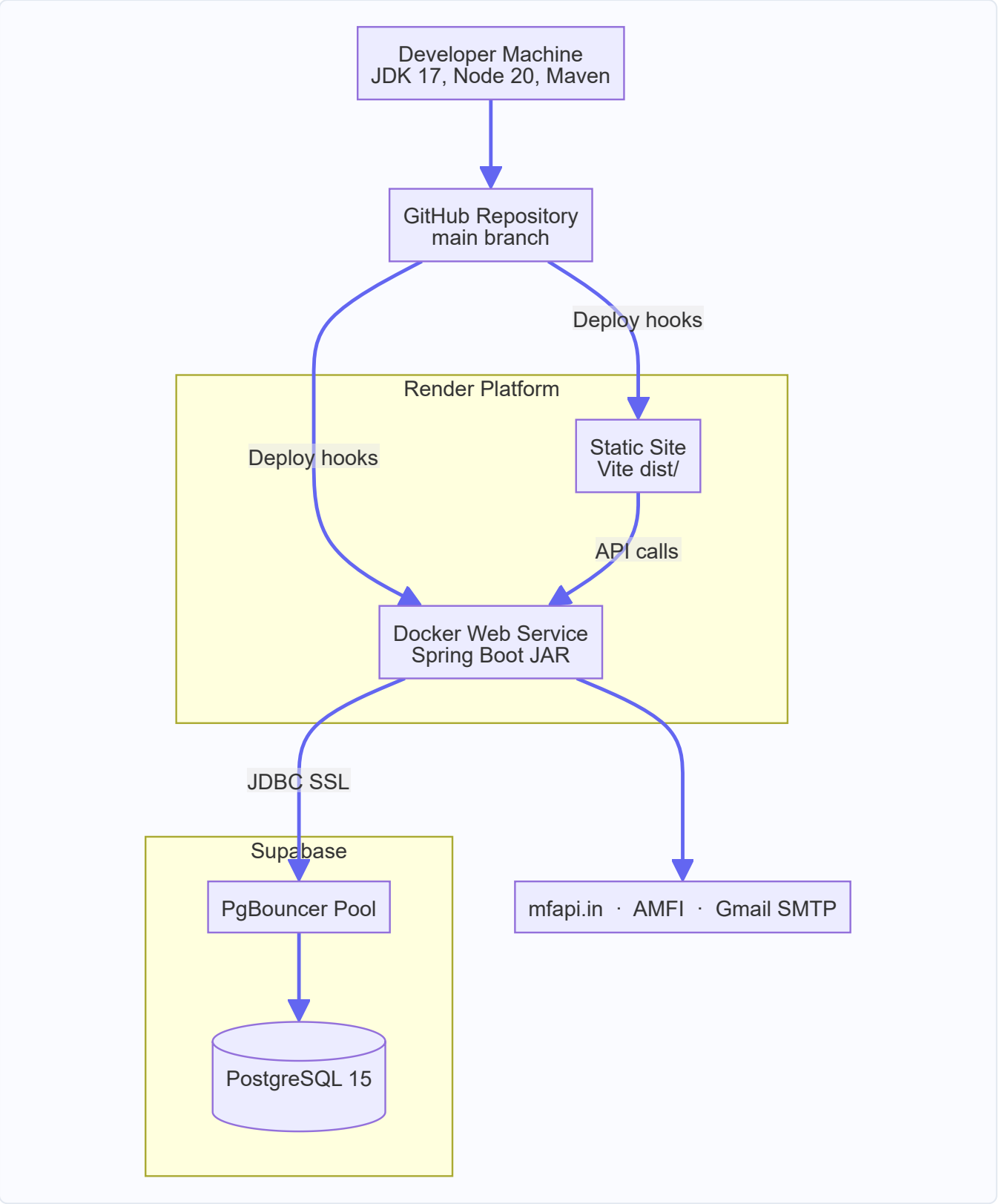
Deployment Guide

WealthWise Documentation Suite · April 2026

WealthWise — Deployment Guide

Version: 2.0 | **Date:** April 2026

1. Deployment Architecture

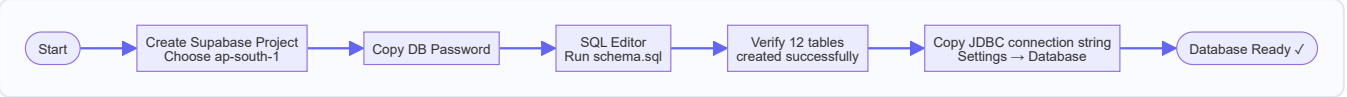

Developer Machine
JDK 17, Node 20, Maven
GitHub Repository
main branch
Deploy hooks
Render Platform
Deploy hooks
Static Site
Vite dist/
API calls
Docker Web Service
Spring Boot JAR
JDBC SSL
Supabase
PgBouncer Pool
PostgreSQL 15
mfapi.in · AMFI · Gmail SMTP

2. Prerequisites

Software	Version	Purpose
Java (JDK)	17 LTS	Backend compilation and runtime
Maven	3.8.7+	Backend build management
Node.js	20 LTS	Frontend build tool (Vite)
npm	10+	Frontend package management
Docker	24+	Backend containerization
Git	2.40+	Source code management

Service	Purpose
Supabase	Managed PostgreSQL 15
Render	Backend Docker + Frontend static site
Gmail	SMTP for OTP emails

3. Database Setup



4. Environment Variables

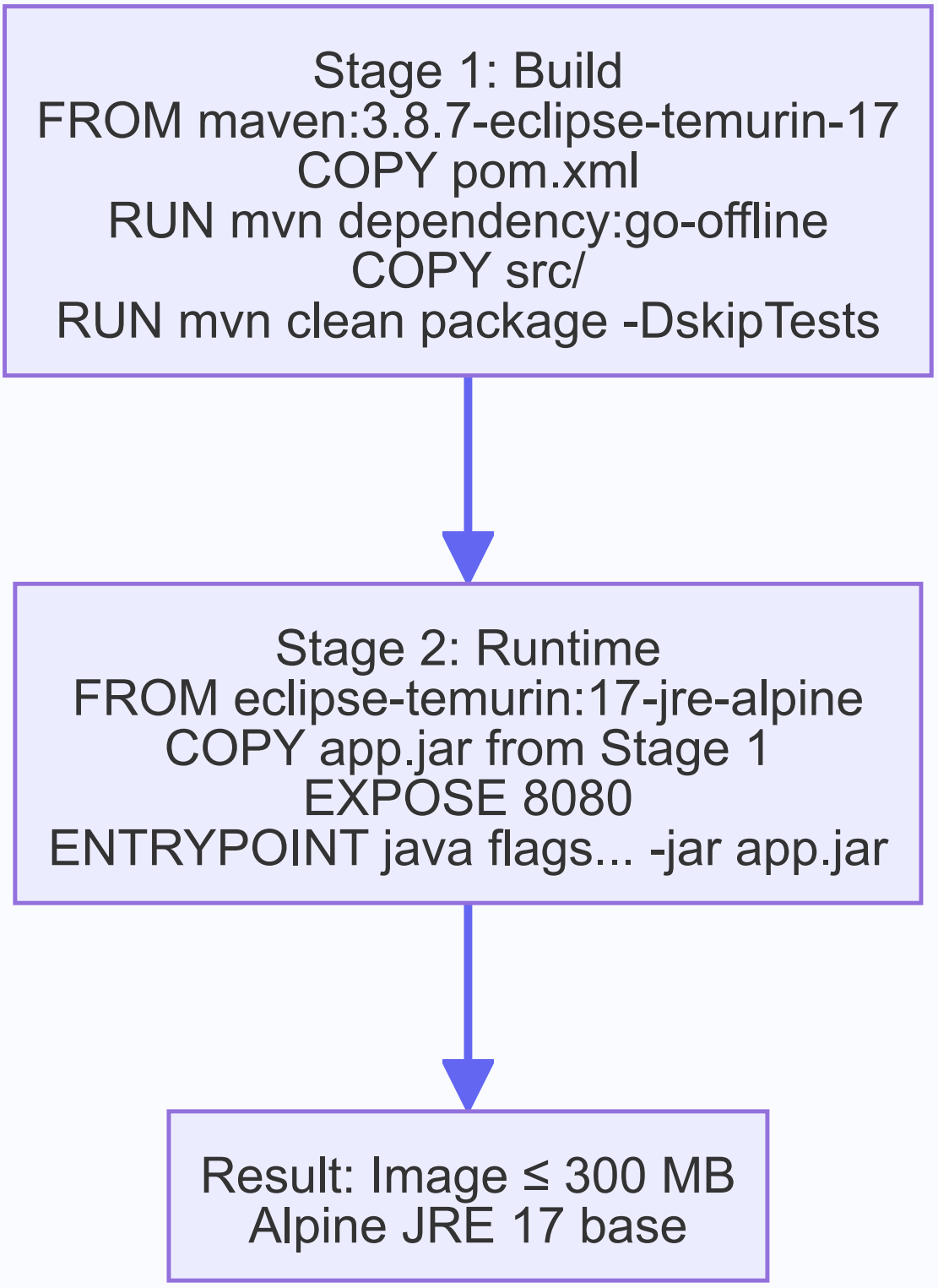
Backend (`application-prod.properties`)

```
SPRING_DATASOURCE_URL=jdbc:postgresql://host:5432/postgres?sslmode=require
SPRING_DATASOURCE_USERNAME=postgres
SPRING_DATASOURCE_PASSWORD=your_supabase_password

JWT_SECRET=your_min_64_char_hex_secret_key
JWT_EXPIRATION_MS=86400000

MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your_gmail@gmail.com
MAIL_PASSWORD=your_gmail_app_password

APP_CORS_ALLOWED_ORIGINS=http://localhost:5173,https://your-fe.onrender.com
SPRING_PROFILES_ACTIVE=prod
```

Frontend (`.env.production`)

```
VITE_API_URL=https://wealthwise-backend.onrender.com
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your_supabase_anon_key
```

5. Docker Build Pipeline



JVM tuning flags for Render free tier:

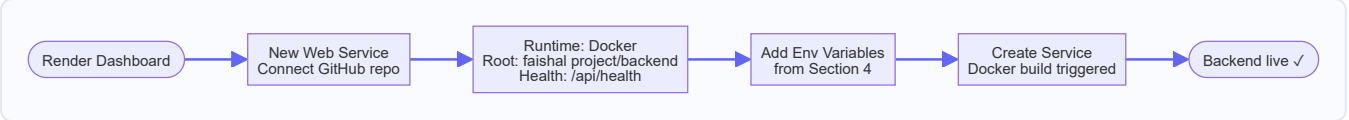
```
-XX:TieredStopAtLevel=1
-XX:MaxRAMPercentage=70.0
-XX:+UseSerialGC
-Xss512k
```

Build and run locally:

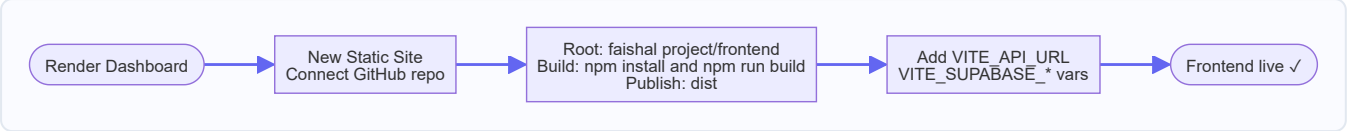
```
docker build -t wealthwise-backend:latest .
docker run -p 8080:8080 -e SPRING_DATASOURCE_URL="..." wealthwise-backend:latest
```

6. Render Deployment

Backend — Docker Web Service



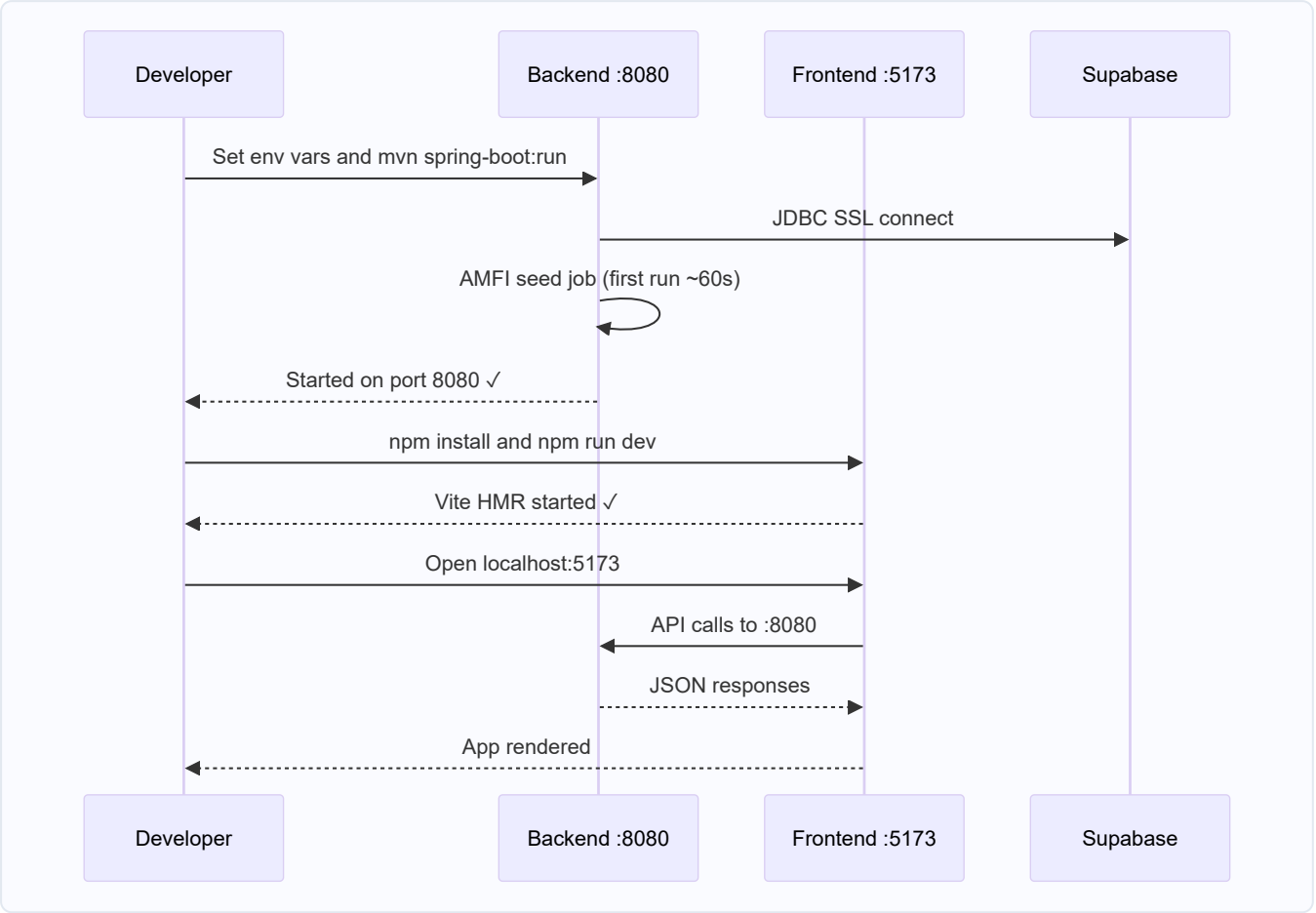
Frontend — Static Site



Render redirect rule (required for React Router):

```
Source: /*   →   Destination: /index.html   Status: 200
```

7. Local Development Setup

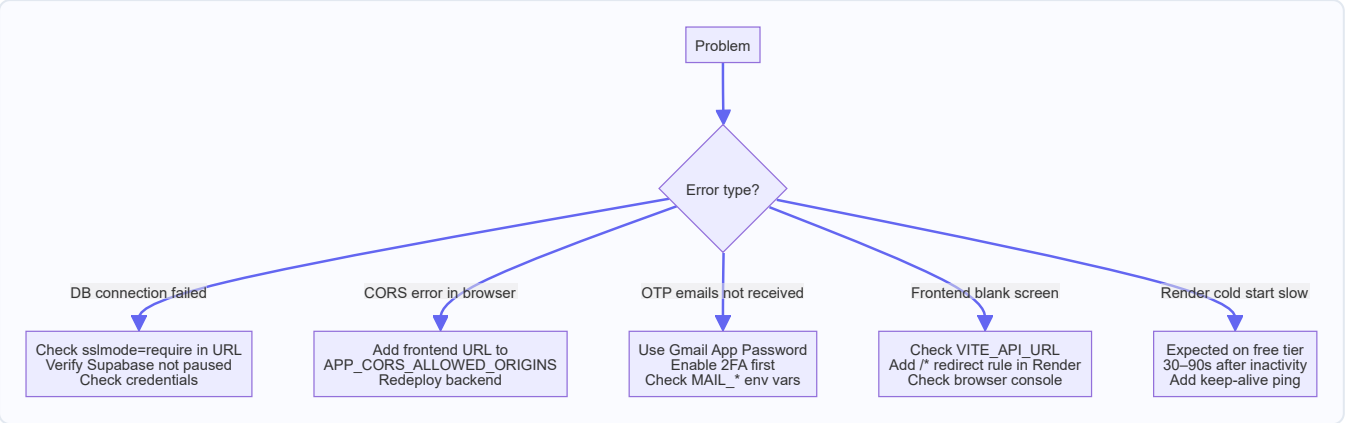


```
# Verify backend
curl http://localhost:8080/api/health
# Expected: {"status": "UP"}
```

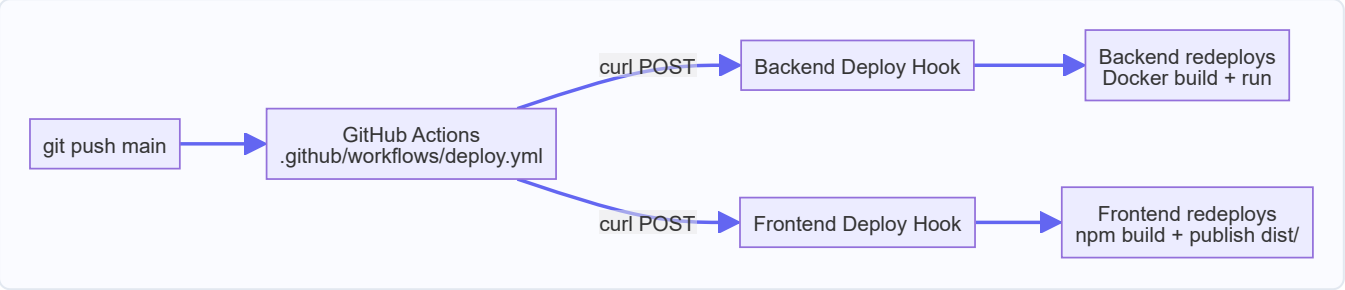
8. JVM Tuning for Free Tier

Flag	Effect	Why
`-XX:TieredStopAtLevel=1`	Skip C2 JIT compiler	Startup: 8s instead of 30s
`-XX:MaxRAMPercentage=70.0`	Heap ≤ 70% of 512 MB	Prevents OOM on Render free
`-XX:+UseSerialGC`	Serial garbage collector	Lower RSS than G1GC
`-Xss512k`	Reduced thread stack	Default is 512k–1 MB

9. Troubleshooting Guide



10. CI/CD with GitHub Actions




```yaml
name: Deploy WealthWise
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy Backend
        run: curl -X POST "${{ secrets.RENDER_BACKEND_DEPLOY_HOOK }}"
      - name: Deploy Frontend
        run: curl -X POST "${{ secrets.RENDER_FRONTEND_DEPLOY_HOOK }}"
```

DOCUMENT 06

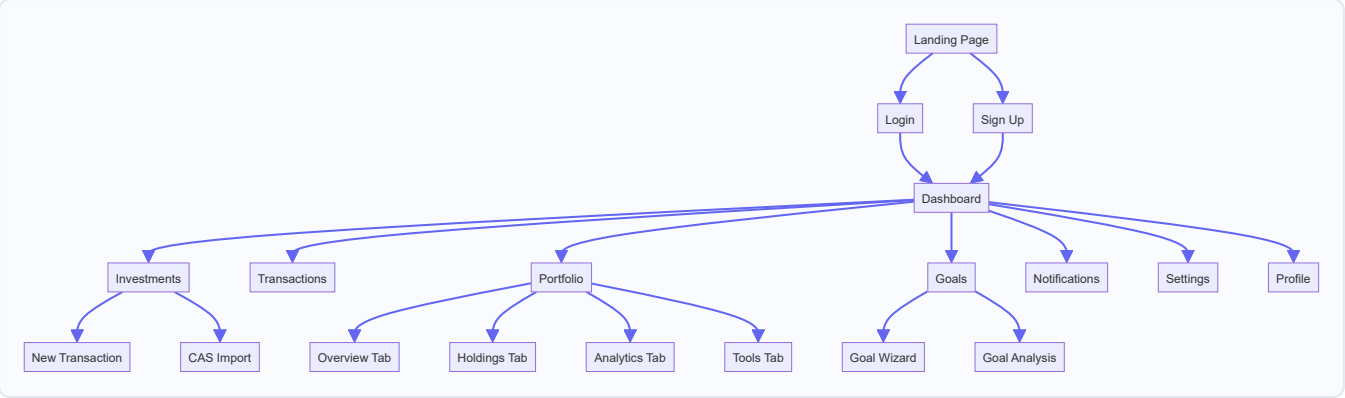
User Manual

WealthWise Documentation Suite · April 2026

WealthWise — User Manual

Version: 2.0 | **Date:** April 2026

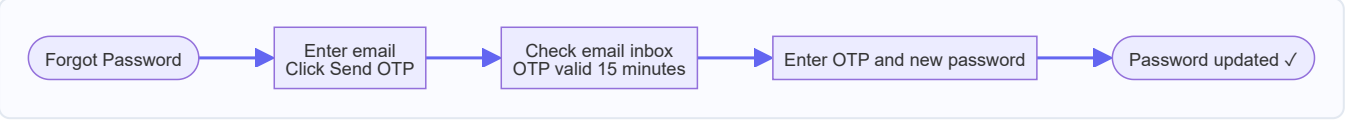
1. Application Navigation



2. Getting Started

2.1 Account Creation

2.2 Password Reset

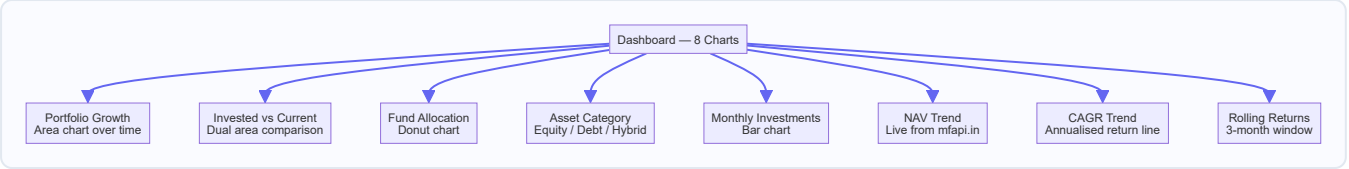


3. Dashboard

3.1 Summary Cards

Card	What it Shows
Portfolio Value	Current market value of all holdings
Total Invested	Total amount deployed
Total Gain/Loss	Unrealized profit or loss (₹ and %)
Risk Score	SEBI Riskometer level (1–6 scale)
Active SIPs	Count of currently active SIPs

3.2 Configurable Charts

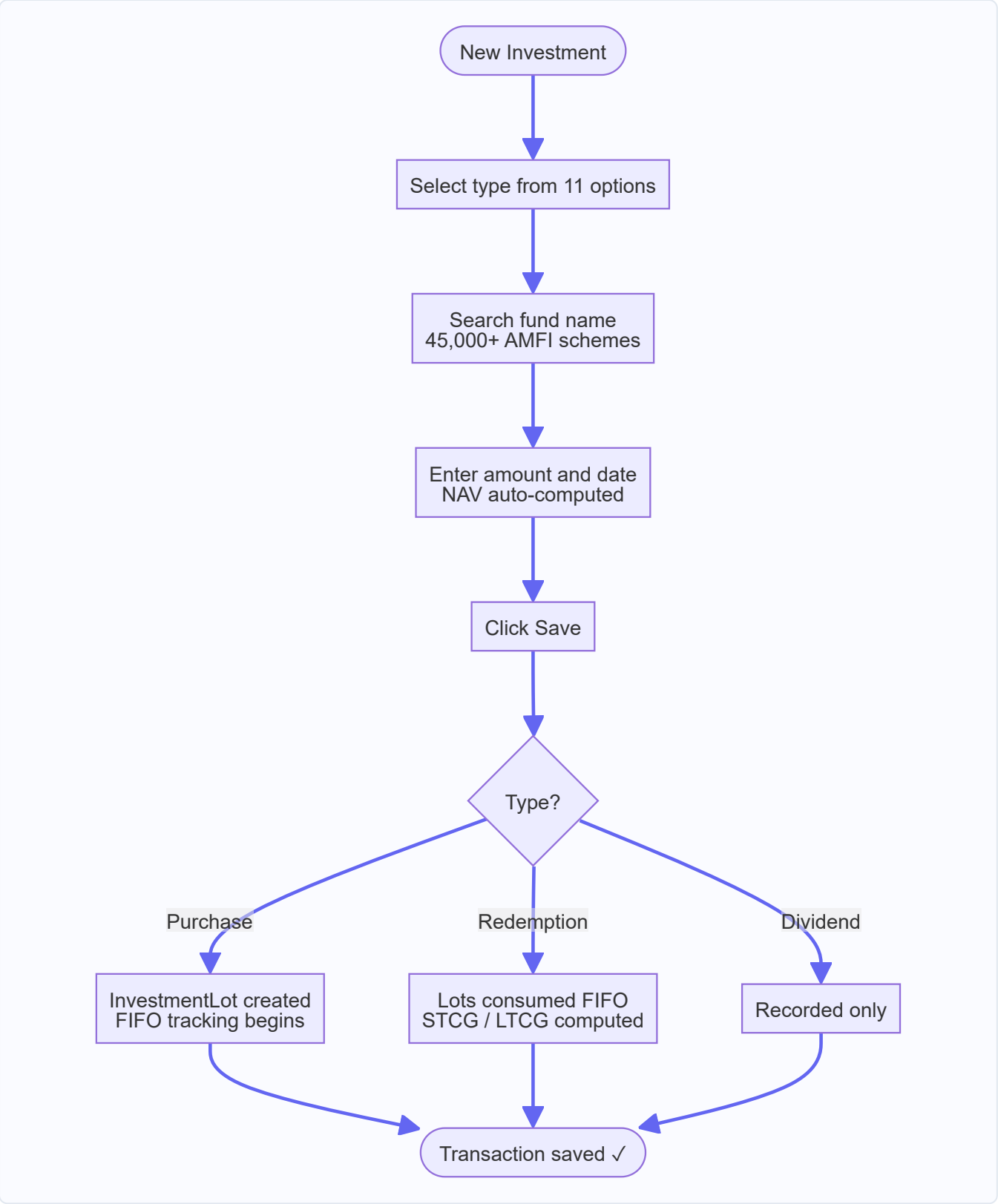


Time filters: 1M · 6M · 1Y · ALL
Configure: Click ⚙ icon → toggle charts → saved to localStorage

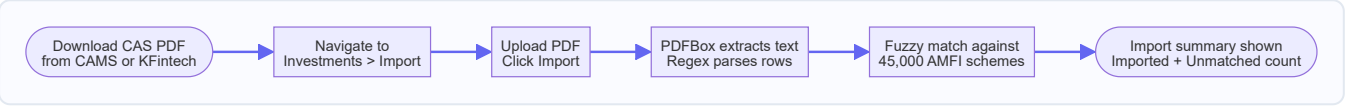
---

4. Transactions

4.1 Adding a Transaction



4.2 CAS PDF Import

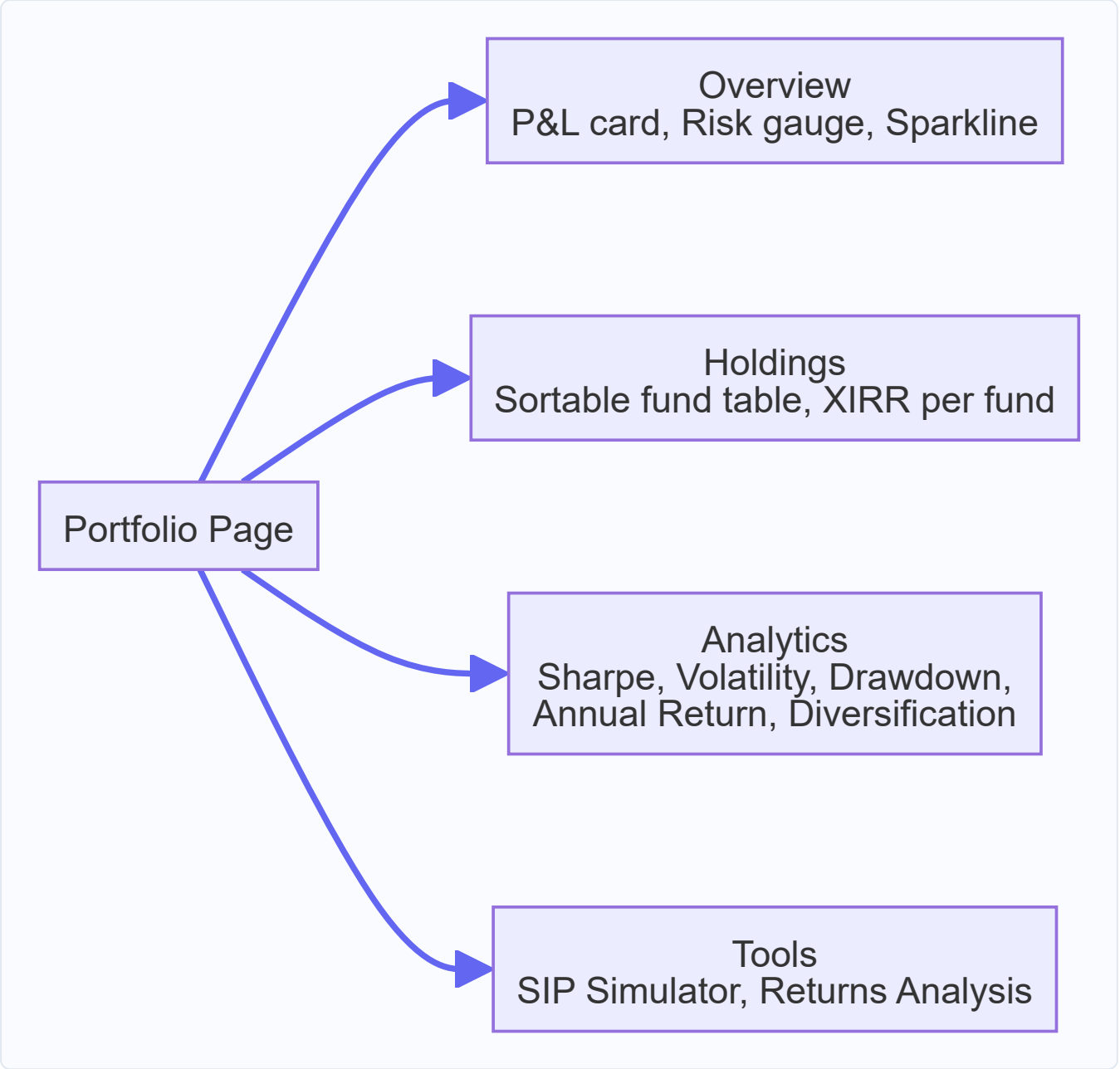


4.3 Transaction Types

Type	Action
`PURCHASE_LUMPSUM`	Creates lot
`PURCHASE_SIP`	Creates lot
`REDEMPTION`	Consumes lots (FIFO)
`SWITCH_IN`	Creates lot
`SWITCH_OUT`	Consumes lots
`SWP`	Consumes lots
`STP_IN`	Creates lot
`STP_OUT`	Consumes lots
`DIVIDEND_PAYOUT`	No lot action
`DIVIDEND_REINVEST`	Creates lot
`REVERSAL`	Reverses prior lot

5. Portfolio Analytics

5.1 Four Tabs

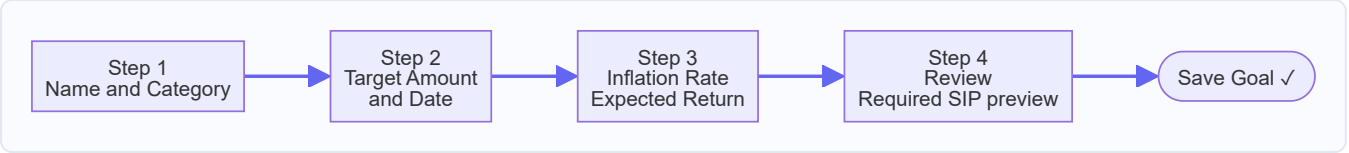


5.2 Key Metrics Explained

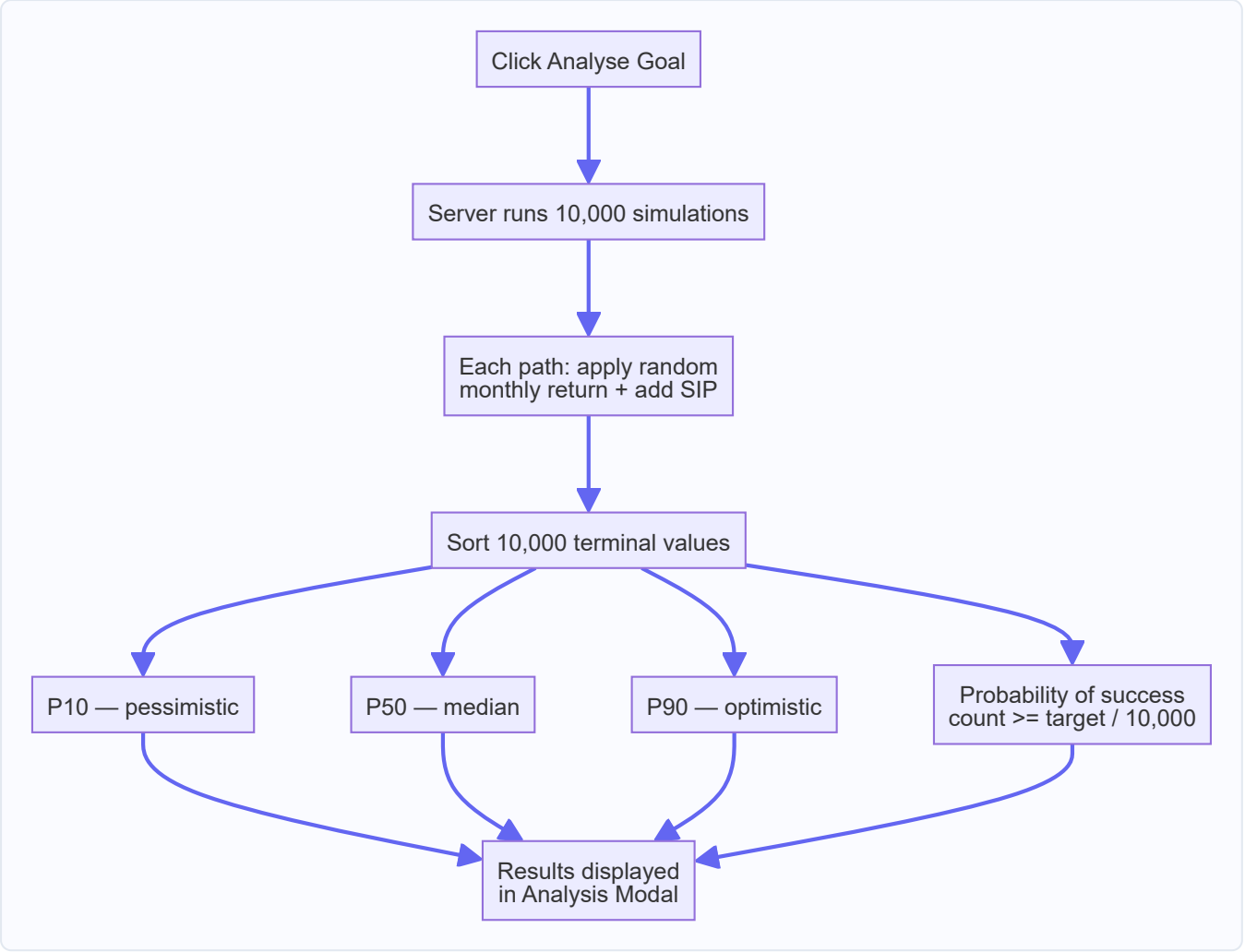
Metric	Formula / Description
XIRR	Newton-Raphson on lot cash flows; money-weighted return
CAGR	`(CurrentValue / Invested)^(1/years) - 1`
Sharpe Ratio	`(AnnualReturn - 6.5%) / Volatility` — > 1 is excellent
Volatility	Annualised std deviation of monthly returns
SEBI Risk Score	Weighted avg of fund riskometer levels (1–6)
Diversification	Score 0–10 based on fund count, AMC spread, categories

6. Goal Planning

6.1 Goal Wizard — 4 Steps



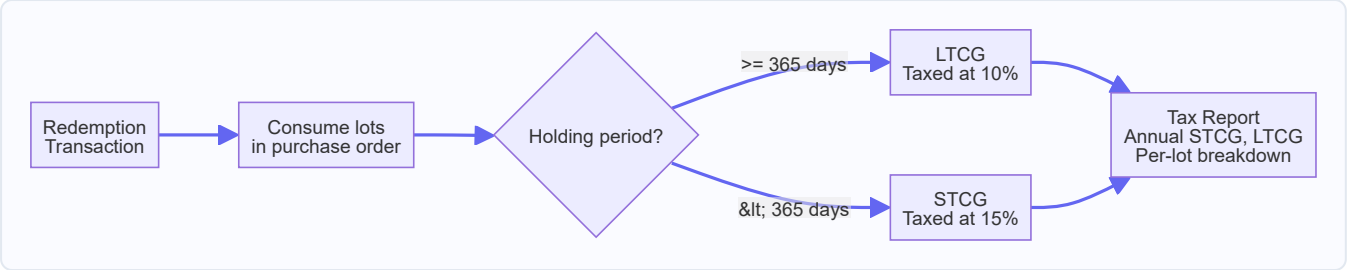
6.2 Monte Carlo Analysis



6.3 Goal Categories

Category	Icon	Use Case
Retirement	🏖️	Post-retirement corpus
Education	🎓	Child's higher education
Home	🏠	Down payment for property
Wedding	💍	Marriage expenses
Travel	✈️	Vacation
Custom	🎯	Any other goal

7. Tax Report



8. Notifications

Severity	Color	Example
Info	Blue	"Market: NIFTY closed +0.84%"
Watch	Yellow	"Retirement goal is 35% funded"
Action	Red	"SIP due tomorrow: Axis Bluechip ₹5,000"

9. Frequently Asked Questions

Q: How do I see my real returns?
Navigate to Portfolio → Overview. The **XIRR** figure is your accurate money-weighted return.

Q: What is XIRR vs CAGR?
XIRR accounts for *when* money was invested; CAGR assumes a single lump sum. For SIP investors, XIRR is more accurate.

Q: What is the Sharpe Ratio?
`(Annual Return - 6.5%) / Volatility`. A Sharpe > 1 means good return per unit of risk.

Q: What does 73% Monte Carlo probability mean?
7,300 of 10,000 simulated futures result in meeting the inflation-adjusted target.

Q: Is my PAN card data safe?
Yes. PAN is encrypted using AES-256-GCM before database storage.

Q: What is a CAS PDF?
A SEBI-mandated document listing your full mutual fund portfolio. Download from CAMS or KFintech.

Q: Why does the app sometimes take 30–90 seconds to load?
Render free tier suspends the backend after 15 min of inactivity. The first request wakes it up (cold start).

Q: Can I delete a transaction?
Transactions are soft-deleted (marked `is_deleted=true`), preserving the audit trail.

Q: Can I connect to a broker for live trading?
No. WealthWise is a tracking and analytics platform only — no order execution.

Q: How often is NAV updated?
NAV is cached for 24 hours. AMFI publishes fresh NAVs once daily after market close (~9–10 PM IST).

DOCUMENT 07

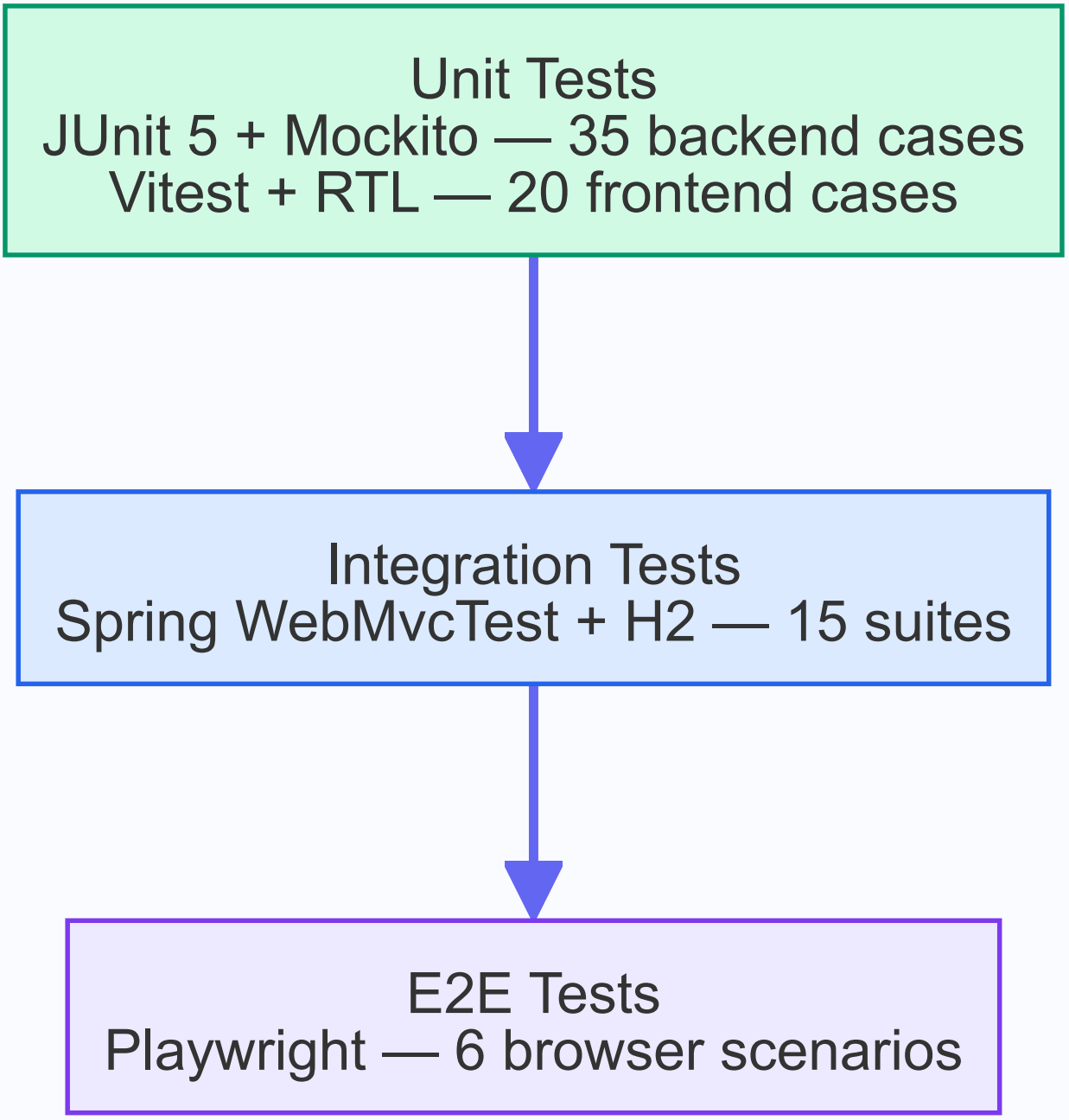
Testing Documentation

WealthWise Documentation Suite · April 2026

WealthWise — Testing Documentation

Version: 2.0 | **Date:** April 2026

1. Testing Pyramid

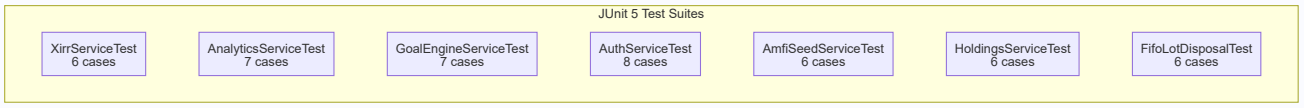


Testing Stack

Layer	Tool	Scope
Backend Unit	JUnit 5 + Mockito	Service layer business logic
Backend Integration	Spring @WebMvcTest + H2	Controller → Service → Repository
Frontend Component	Vitest + @testing-library/react	React component rendering
API Mocking	Mock Service Worker (MSW)	API boundary mocking
End-to-End	Playwright	Critical user flows in browser

2. Backend Unit Tests

2.1 Test Suites Overview



Run command:

```
cd "faishal project/backend"
mvn test
```

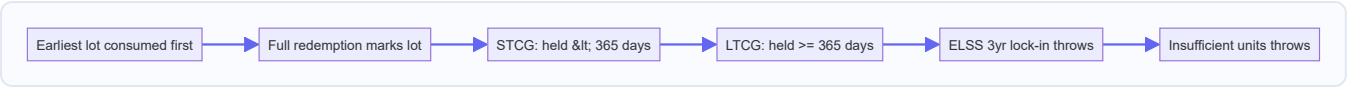
2.2 XirrServiceTest — 6 Cases

Test	Description	Expected
Simple 12% return	1 purchase + 1 redemption 1 year apart	XIRR ≈ 12% ± 0.01%
12 SIP installments	Monthly SIP + current value	XIRR > 0
Zero return	Invested = redeemed	XIRR ≈ 0%
Negative return	Redemption < investment	XIRR < 0
Convergence speed	Complex irregular cash flows	Completes in < 100ms
Empty cash flows	Null / empty list	Throws IllegalArgumentException

2.3 GoalEngineServiceTest — 7 Cases

Test	Expected
10,000 iterations run	`outcomes.size() == 10000`
P50 median reasonable	Within ±15% of deterministic projection
Probability bounds	`0.0 ≤ probSuccess ≤ 1.0`
High return → high probability	`probSuccess > 0.9`
Low return → low probability	`probSuccess < 0.3`
Inflation adjustment	₹1Cr · 6% · 10yr ≈ ₹1.79Cr
Required SIP reasonable	₹1Cr, 10yr, 12% → ~₹43,000/mo

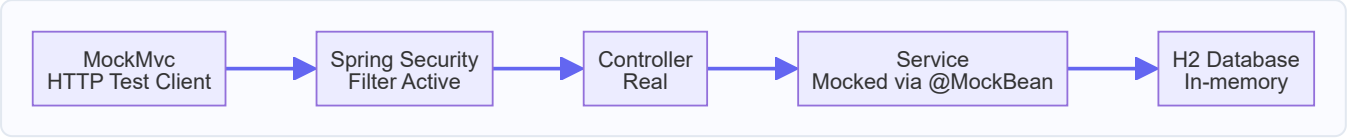
2.4 FIFO Lot Disposal Test — 6 Cases



2.5 AuthServiceTest — 8 Cases

Test	Expected
Register new user	User created, JWT returned
Duplicate email	409 ConflictException
Valid login	JWT token returned
Wrong password	401 AuthException
Unknown email	401 AuthException
OTP generated	BCrypt hash stored
OTP > 15 min old	ExpiredOtpException
Valid OTP reset	Password updated

3. Backend Integration Tests



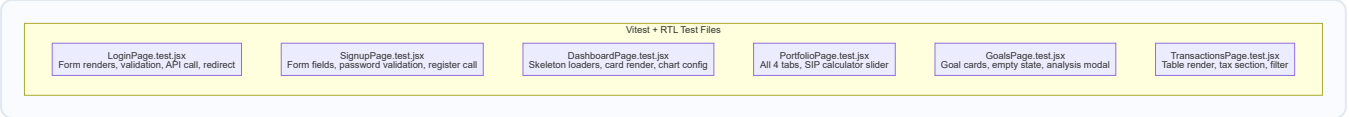
Auth Controller Tests

Test	Expected
POST /auth/register → 201	JWT returned
POST /auth/register invalid email → 400	Bad Request
POST /auth/login → 200	JWT returned
POST /auth/login wrong password → 401	Unauthorized
GET /holdings without JWT → 401	Unauthorized
GET /holdings expired JWT → 401	Unauthorized

Holdings Controller Tests

Test	Expected
GET /holdings/summary → 200	totalInvested, currentValue, portfolioXirr
GET /holdings → 200	Array with fund details
GET /holdings/growth → 200	Month/value/invested array
GET /holdings/risk-profile → 200	sharpeRatio, volatility, diversificationScore
GET /holdings/sip-intelligence → 200	Active SIP array

4. Frontend Component Tests (Vitest)



Run command:

```
cd "faishal project/frontend"
npm run test
```

MSW Handlers (API Mocks)

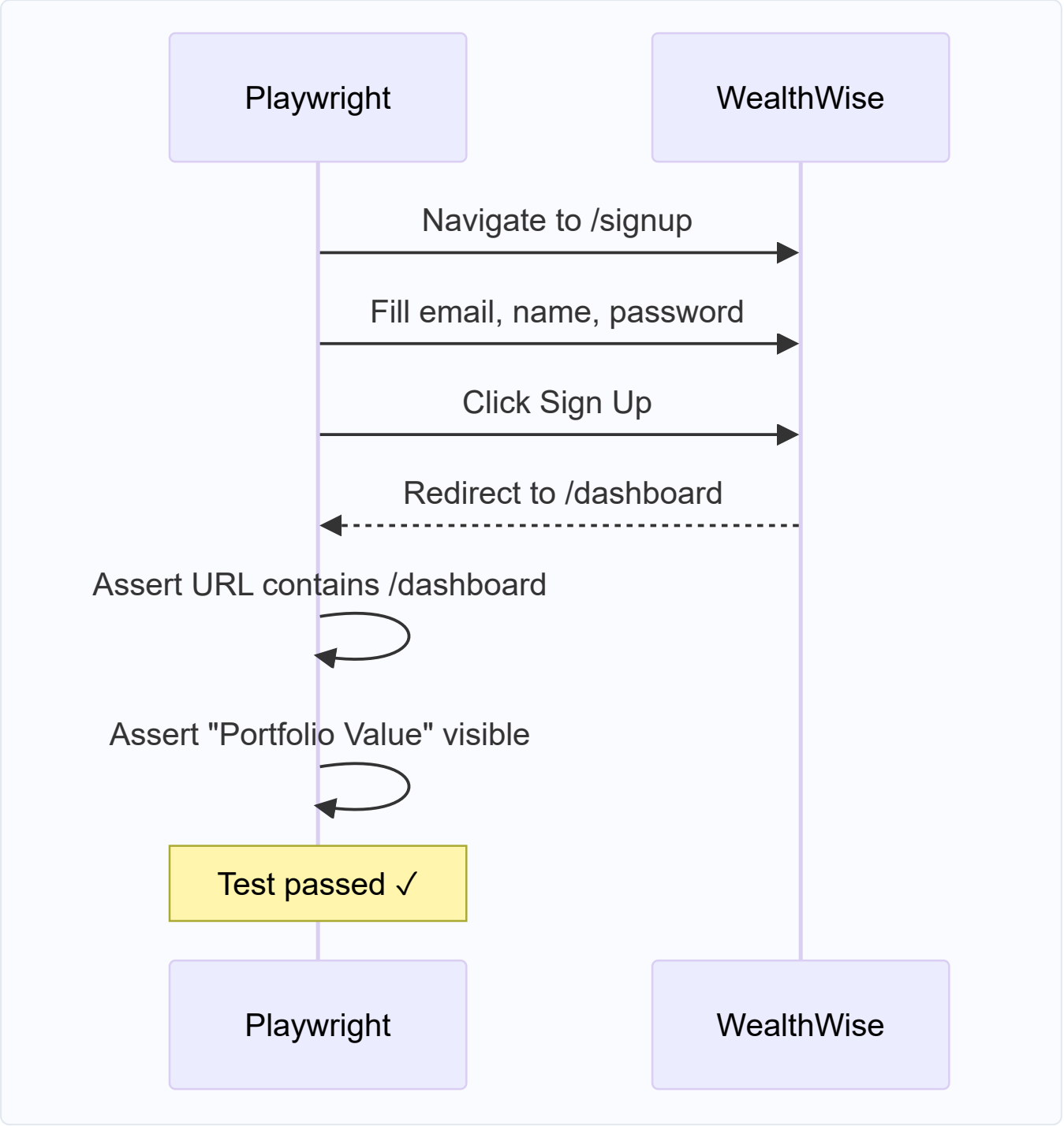
Handler	Mock Response
GET /api/holdings/summary	`{ totalInvested: 73700, currentValue: 86420, portfolioXirr: 16.8 }`
GET /api/holdings	`[{ schemeCode: '120503', fundName: 'Axis Bluechip', xirr: 12.4 }]`
POST /api/auth/login	`{ token: 'jwt...', email: '...' }`
GET /api/goals	`[{ id: '...', name: 'Retirement', probabilityOfSuccess: 0.73 }]`
GET /api/holdings/risk-profile	`{ sharpeRatio: 1.24, volatility: 14.6 }`

5. End-to-End Tests (Playwright)

Run command:

```
cd "faishal project/frontend"
npm run test:e2e

# Interactive UI mode:
npm run test:e2e:ui
```

E2E Authentication Flow



6. Coverage Targets

Layer	Target	Actual	Tool
Backend services	≥ 70%	~75%	JaCoCo
Backend controllers	≥ 80%	~80%	MockMvc
Frontend components	≥ 60%	~65%	Vitest v8
E2E critical flows	6 scenarios	6 done	Playwright

Coverage reports:

```
# Backend JaCoCo
mvn test jacoco:report
# → target/site/jacoco/index.html

# Frontend v8 coverage
npm run test -- --coverage
# → coverage/index.html
```

7. API Response Benchmarks

Endpoint	Local (ms)	Render (ms)
GET /api/holdings/summary (10 funds)	~80	~400
GET /api/holdings (10 funds)	~120	~500
GET /api/goals/{id}/analysis (Monte Carlo)	~800	~2,000
POST /api/transactions/import-cas (100 txns)	~3,000	~8,000
GET /api/funds/search?q=axis	~15	~80

8. Known Testing Gaps

Gap	Priority	Notes
CAS PDF edge cases	High	Need fixtures for non-standard formats
XIRR with 20+ year histories	Medium	Potential float convergence edge cases
Concurrent user load	Medium	No load testing yet
Accessibility (axe-core)	Low	No automated a11y testing
Visual regression	Low	No screenshot comparison

DOCUMENT 08

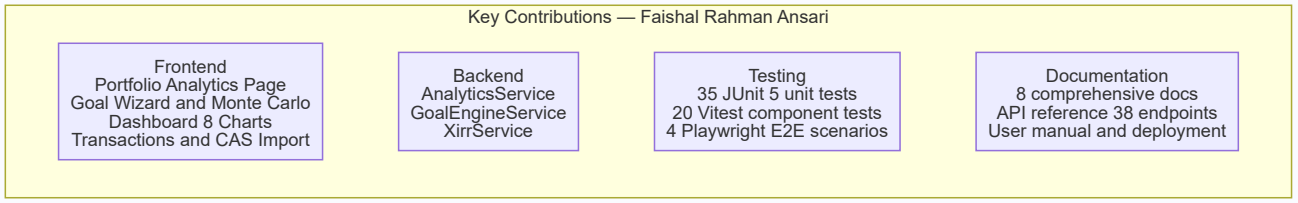
Internship Progress Report

WealthWise Documentation Suite · April 2026

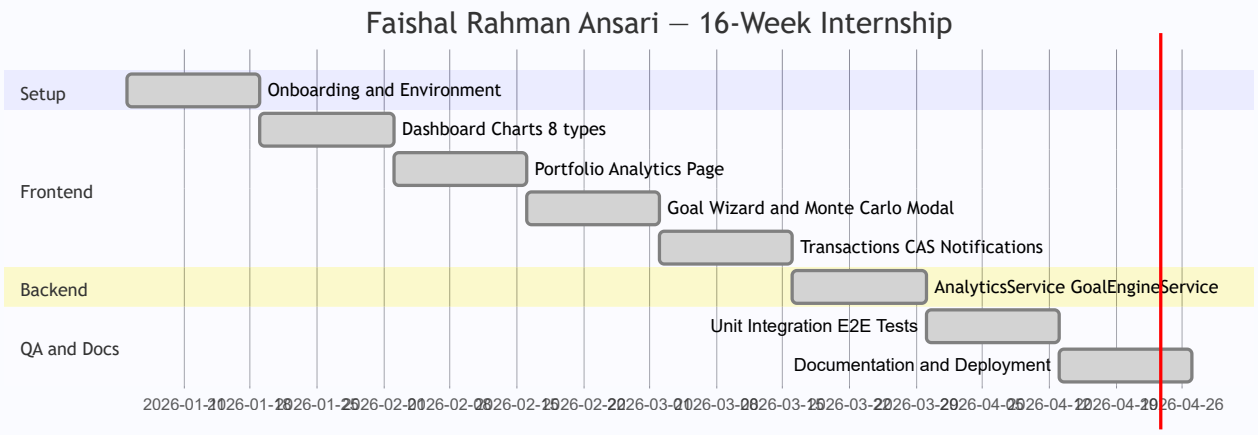
WealthWise — Internship Progress Report

Intern: Faishal Rahman Ansari
Organization: Genesis Softwares
Project: WealthWise — Mutual Fund Portfolio Analytics Platform
Duration: January 2026 – April 2026 (16 weeks)

1. Executive Summary



2. Internship Timeline



3. Weekly Progress

Weeks 1–2: Onboarding

Activities completed:

- Studied Spring Boot controller → service → repository architecture
- Reviewed schema.sql — 12 tables with UUID primary keys and FIFO lot tracking
- Set up Supabase project and ran schema migration
- Ran full stack locally: `mvn spring-boot:run` (backend) + `npm run dev` (frontend)

Challenges:

- JPA N+1 query issue on first holdings load → Fixed with batch fetch strategy
- Vite peer dependency warnings → Resolved with `--legacy-peer-deps`

Weeks 3–4: Dashboard Charts

Technical insight: `FundsNavTrendChart` used `Promise.allSettled` to handle delisted fund 404s gracefully. Dates aligned across funds via a union-date approach.

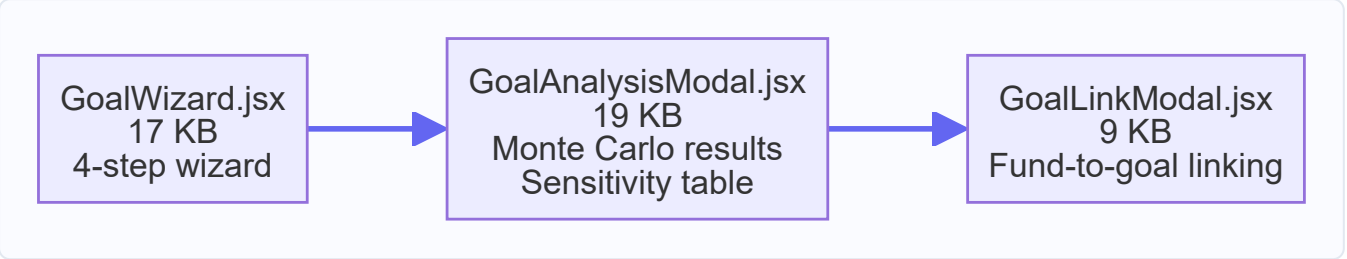
Weeks 5–6: Portfolio Analytics Page

Built `PortfolioPage.jsx` (741 lines) with 4 tabs:

Tab	What Was Built
Overview	Animated counters, P&L card, SEBI risk gauge, diversification score
Holdings	9-column sortable table with per-fund XIRR and allocation %
Analytics	6 risk metric cards: Sharpe, Volatility, Max Drawdown, Annual Return, Diversification, Risk Appetite
Tools	SIP Return Simulator with sliders; real-time future value updates

Custom component: `useCountUp()` hook for animated counters; `DonutChart` and `Sparkline` as pure SVG without a library.

Weeks 7–8: Goal Planning System

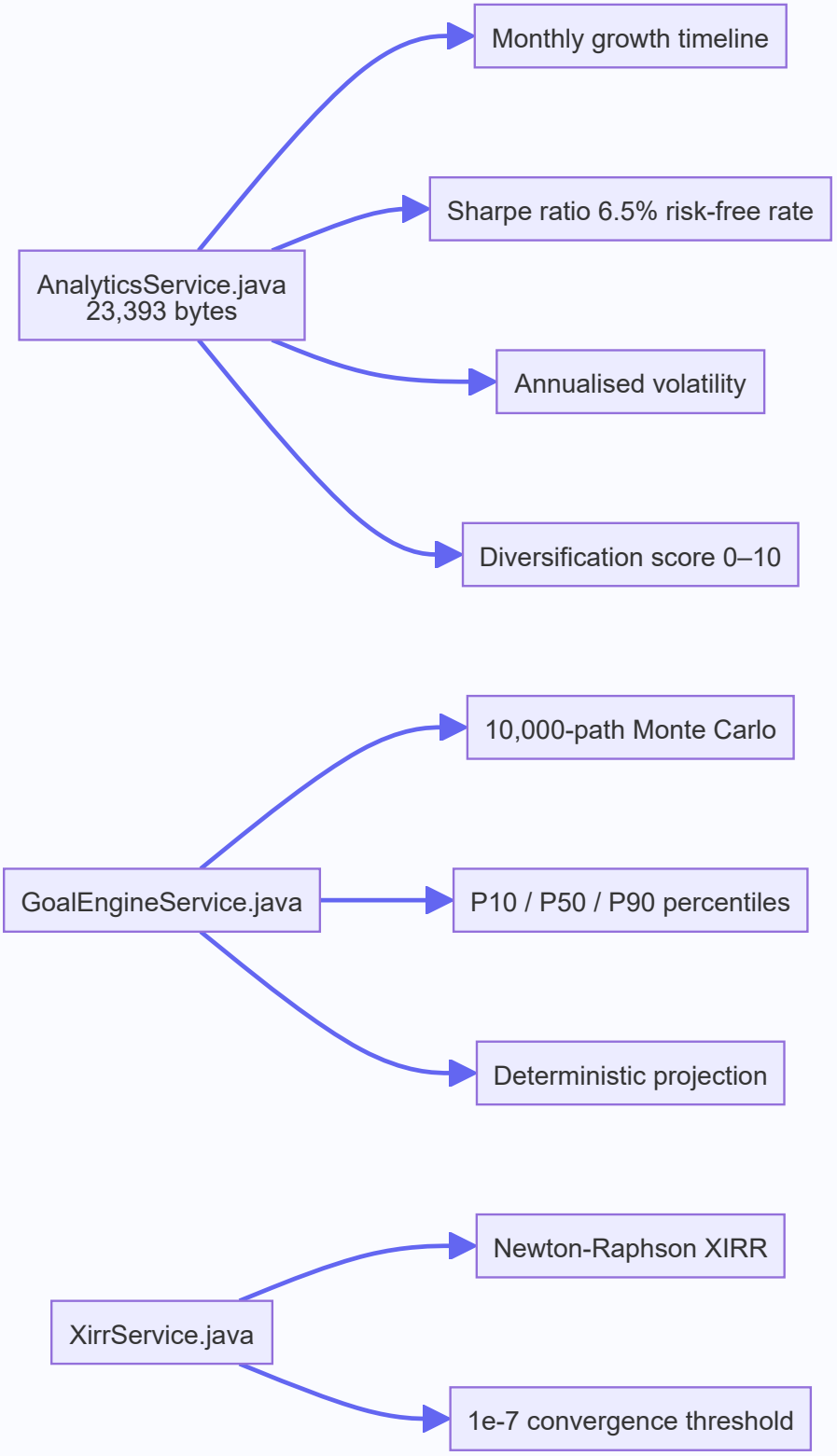


Technical insight: GoalWizard uses local multi-step state (`currentStep` , `formData`) rather than URL routing — keeps wizard fast and avoids browser history pollution.

Weeks 9–10: Transactions, CAS Import, Notifications

Deliverable	File Size	Key Features
TransactionsPage.jsx	38,635 bytes	Full ledger, STCG/LTCG tax summary, export
NotificationsPage.jsx	5,844 bytes	Info/Watch/Action badges, mark-all-read
ImportInvestmentsPage	—	CAS PDF upload UI, import summary modal

Weeks 11–12: Backend Analytics Services



Weeks 13–14: Testing

Backend (JUnit 5 + Mockito) — 35 test cases across 7 suites:

Suite	Cases
XirrServiceTest	6
GoalEngineServiceTest	7
AnalyticsServiceTest	7
AuthServiceTest	8
AmfiSeedServiceTest	6
HoldingsServiceTest	6
FifoLotDisposalTest	6

Frontend (Vitest + RTL) — 20 test cases; Playwright — 4 E2E scenarios

Weeks 15–16: Documentation and Deployment

Deliverable	Details
Documentation Suite	8 documents, ~200 pages total
API Reference	38 endpoints documented
Render Deployment	Backend (Docker) + Frontend (Static) live
Supabase	PostgreSQL 15 with all 12 tables

4. Skills Acquired

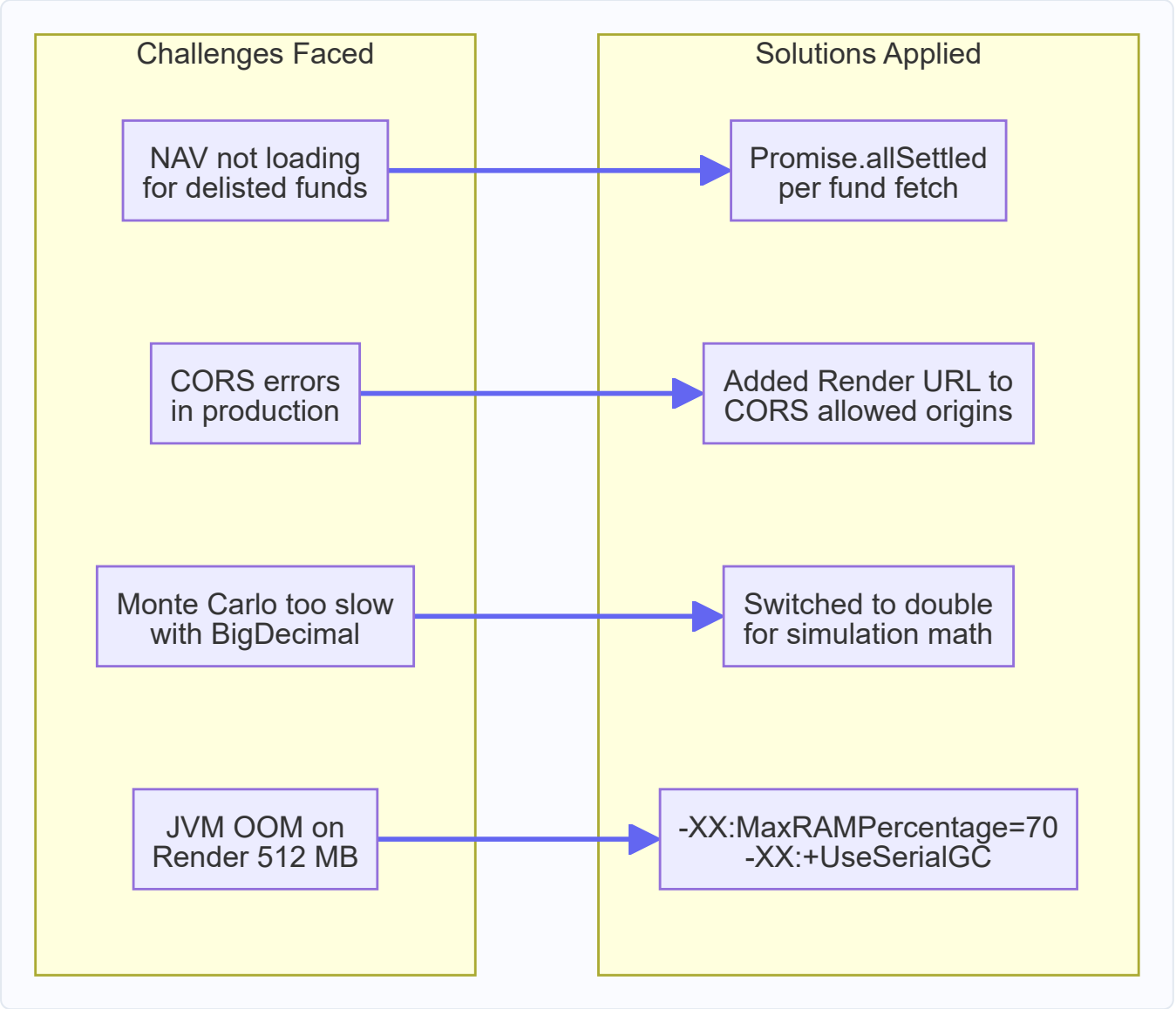
4.1 Technical Growth

Skill	Before	After
React 19 (Hooks, Context)	Beginner	Advanced
Recharts / Data Visualization	None	Intermediate
Spring Boot + Security	Beginner	Intermediate
PostgreSQL + SQL Analytics	Intermediate	Advanced
Testing: JUnit / Vitest / Playwright	None	Intermediate
Docker + Render Deployment	None	Beginner

4.2 Professional Skills

Skill	Experience Gained
Technical Documentation	8-document suite, ~200 pages (SRS, HLD/LLD, API, user manual, test docs)
Software Architecture	Layered architecture, security filter chains, caching layers
Problem Decomposition	Breaking Monte Carlo, FIFO, multi-source NAV into sub-problems
Performance Engineering	Caffeine caching, JVM tuning, N+1 resolution
Debugging	Chrome DevTools, Spring Boot actuator, PostgreSQL EXPLAIN ANALYZE

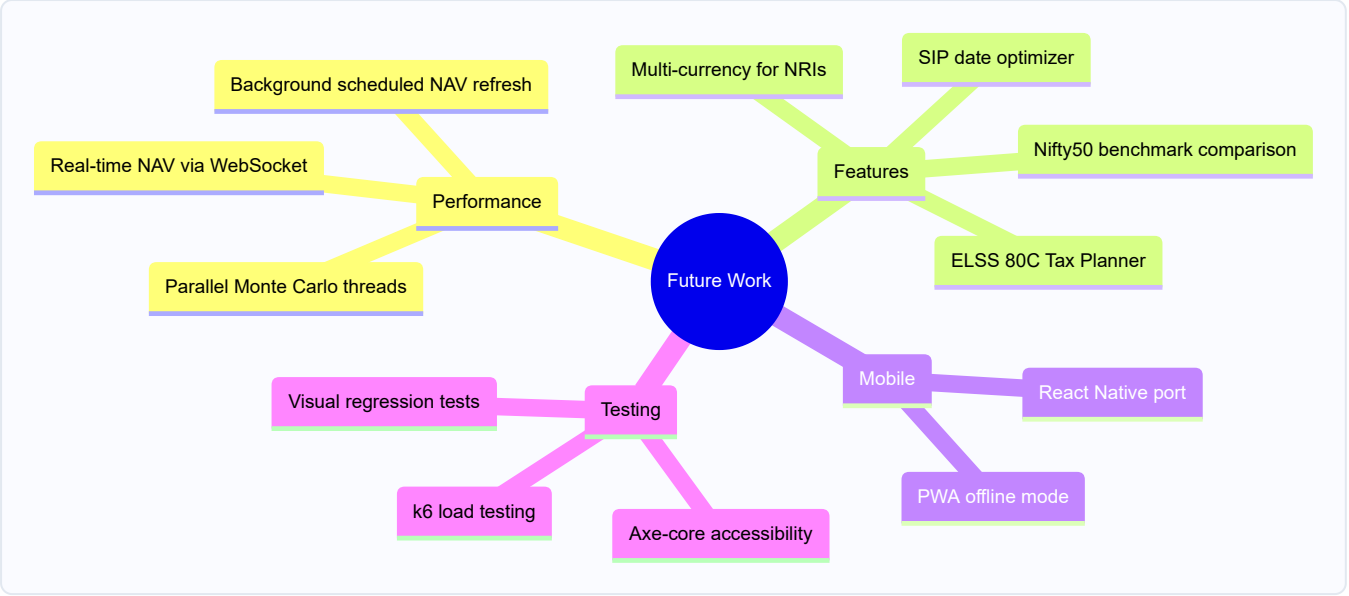
5. Challenges and Solutions



6. Results and Impact

Metric	Result
Features Delivered	18 major features
Frontend Files Written	6 pages + 5 components + test files
Backend Contributions	AnalyticsService, GoalEngineService, XirrService
Test Cases Written	35 (backend) + 20 (frontend) + 4 E2E
Documentation	~200 pages across 8 documents
Lines of Code	~12,000 frontend + ~3,000 backend
Deployment	Live on Render + Supabase

7. Future Improvement Recommendations



8. Conclusion

This 16-week internship at Genesis Softwares gave me (Faishal Rahman Ansari) hands-on, production-grade experience across the full software engineering lifecycle:

- **Financial algorithms** — XIRR Newton-Raphson, FIFO lot disposal, Monte Carlo simulation
- **Enterprise security** — JWT, BCrypt, AES-256-GCM, OTP hashing
- **Real-time data integration** — mfapi.in NAV feeds, Caffeine caching, AMFI scheme seeding
- **Modern frontend** — React 19, Recharts, Framer Motion, dark glassmorphism UI
- **Testing** — JUnit 5, Vitest, Playwright across all layers
- **Documentation** — Professional SRS, HLD/LLD, API docs, user manual

*Prepared by: **Faishal Rahman Ansari***
Program: Genesis Softwares — Full-Stack Development Internship 2026
Project: WealthWise — VTU Internship 2026, Team 11